A DBMS AND JAVA MINI PROJECT REPORT ON

NABTA

E-Commerce and Order Management System

Database Design • Java • Mobile Application



Submitted by

Rasheed Mukhtar

PRN: 25070122152

Roc Mukendi

PRN: 25070122161

Puneet lath

PRN: 25070122144

Under the Guidance of

Prof. Akash Bhattacharyya & Dr.Rashmi Dharwadkar & Dr Jitendra Rajpurohit

Department of Computer Science
Symbiosis Institute of Technology, Pune
Academic Year 2026–2027

INDEX

This report follows the section order of the supplied sample report and adds the database-analysis and Java-concept material required by the two evaluation rubrics.

Section	Title
1.	Introduction
2.	Problem Statement
3.	System Architecture and Modules
4.	Functional Requirements
5.	Entities, Relationships and Attributes
6.	ER and EER Diagrams
7.	Relational Schema
8.	Application of Codd's Rules
9.	Functional Dependencies
10.	Anomaly Analysis
11.	Normalization
12.	MySQL Implementation
13.	SQL Queries, Functions, Procedures and Triggers
14.	Application of Java Concepts
15.	Testing Plan and Demonstration
16.	Limitations and Future Enhancements
17.	Conclusion
Appendix A	Screenshot Capture Guide
Appendix B	Viva Preparation Questions

DOCUMENT EDITING KEY: Orange text on a pale-yellow box marks an image or screenshot that must be inserted. Replace every placeholder before submission.

1. Introduction

Nabta is a bilingual commerce application designed to help a company present and manage its products through a mobile-first interface. Customers can browse categories and products, view product details, maintain a preference list, manage saved delivery addresses, and submit orders. The system also provides a protected administrative workflow for the Owner, Supervisors, and Delivery personnel.

The application uses a centralized Java backend and a MySQL relational database. The Android client is built using HTML, CSS, JavaScript, Ionic, and Capacitor; it communicates with the backend through REST APIs. Firebase Cloud Messaging (FCM) supports relevant order and delivery notifications. A JSP-based web interface is included in the academic implementation to demonstrate the Java web-GUI requirement, while the Ionic application remains the customer mobile interface.

Nabta stores product and category names in English and Arabic simultaneously. The interface can change the order in which the two names appear according to the selected language, while keeping both names visible. Product prices are represented in Saudi Riyals (SAR). The current checkout requirement supports Cash on Delivery (COD); a real online payment provider is reserved for future integration and must not be represented as connected until implemented.

The database model covers product catalogue data, user accounts, carts, preferences, addresses, orders, order items, delivery assignments, payment status records, order-status history, and employee activity. The report explains the data model, relational mapping, functional dependencies, normalization, SQL operations, and the Java concepts required by the supplied evaluation criteria.

1.1 Objectives

- Provide a simple, mobile-friendly catalogue for Nabta products and categories.
- Support bilingual English/Arabic names while preserving readable layout and RTL behaviour.
- Store application data centrally in MySQL through the Java backend.
- Support secure customer accounts, saved addresses, preferences, carts, and orders.
- Give the Owner control over catalogue data, accounts, staff roles, order oversight, statistics, and activity history.
- Allow Supervisors to process orders and assign full or partial deliveries.
- Allow assigned Delivery users to accept assignments and confirm completion.
- Demonstrate the database-design and Java concepts specified by the project evaluation rubrics.

1.2 Scope

The project scope includes catalogue management, bilingual data, customer account functions, cart operations, COD checkout, role-based order processing, delivery assignment, notifications, reporting, and database auditing. Online payment processing is not enabled in this version. A payment transaction model may retain fields needed for future provider integration, but no successful online payment may be simulated or reported.

2. Problem Statement

A commerce business needs a reliable way to present products, maintain accurate inventory, receive customer orders, coordinate employees, and keep customers informed about order progress. If these activities are managed manually or through disconnected tools, product information can become inconsistent, orders may be missed, and responsibility for status changes may be difficult to trace.

Nabta addresses these problems through a centralized application in which product and category data are stored once and retrieved by the clients. Customers can access product names in both English and Arabic, choose delivery addresses, and track their orders. The Owner and Supervisors can manage order processing and delivery assignments, while the Delivery role records accepted assignments and completed deliveries. Order-status history and employee activity records provide an audit trail.

The design also addresses data integrity. Primary keys uniquely identify records, foreign keys express relationships, constraints protect essential values, and service-level authorization prevents customers or employees from performing operations outside their roles. The relational design aims to reduce duplication and update anomalies while retaining historical order-item information needed for receipts and audit purposes.

2.1 Proposed Solution

- Use one authoritative MySQL database for catalogue, customer, cart, and order information.
- Use a Java Spring Boot REST API as the business and security boundary.
- Use role-based authorization for Owner, Supervisor, Delivery, and Customer functions.
- Store bilingual fields for names and descriptions that need to be readable in both languages.
- Record every important order-status change with its responsible user and timestamp.
- Use persistent records for delivery assignments and the quantities sent in each partial delivery.
- Keep online payment options disabled until a provider is integrated and payment verification has been tested.

3. System Architecture and Modules

3.1 System Architecture

Nabta follows a layered client-server architecture. The mobile client and the academic JSP demonstration interface communicate with the Java backend. The backend performs authentication, validation, authorization, business rules, and database access. MySQL stores the authoritative application records. FCM delivers push notifications to registered devices. Product images are stored in a persistent storage location and referenced by a URL or storage key.

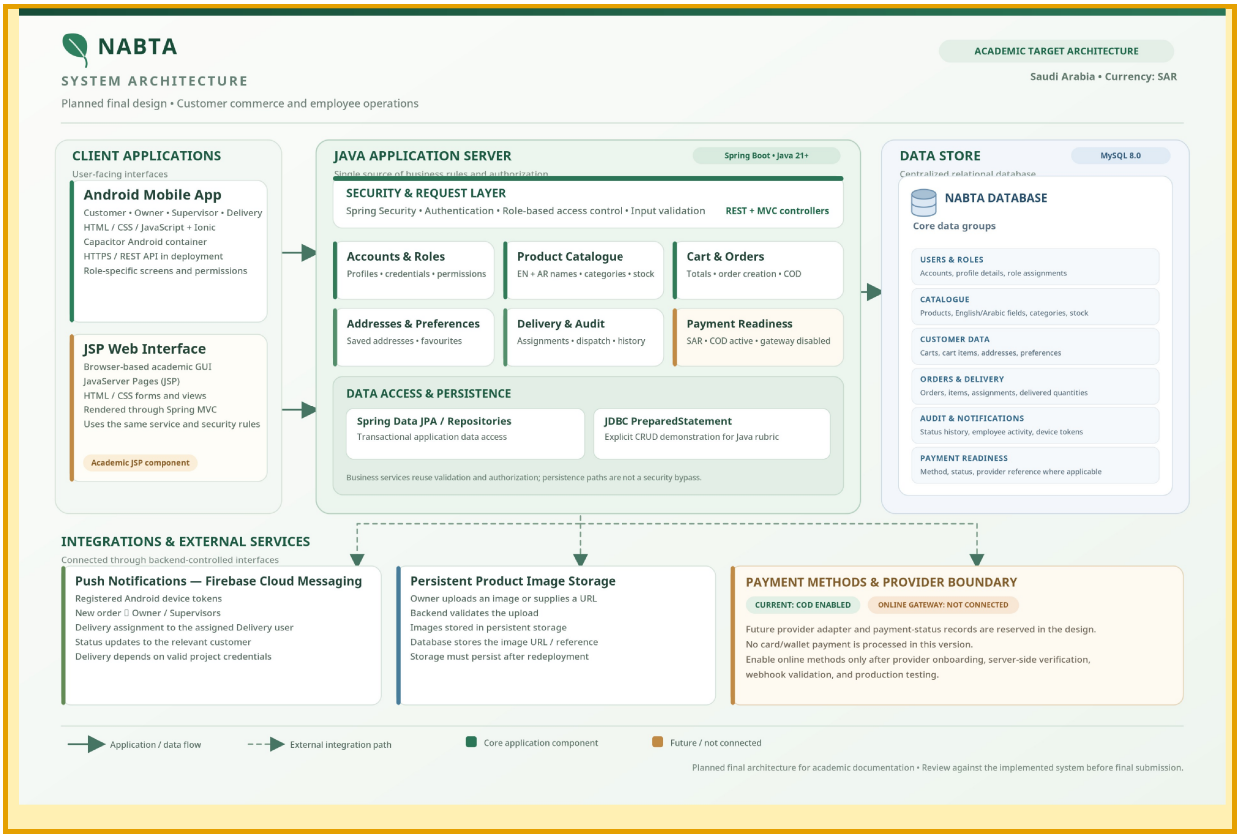


Figure 1. System architecture of the Nabta application.

3.2 Architecture Layers

Layer	Technology / Component	Responsibility
Presentation	Ionic, HTML, CSS, JavaScript, Capacitor	Customer and staff screens, navigation, language direction, cart interface, and mobile access.
Academic web GUI	JSP pages served through the Java web application	Demonstrates server-rendered Java web GUI functions such as product listing or CRUD for the Java rubric.
API and security	Java Spring Boot, Spring Security, REST controllers	Receives API requests, authenticates users, applies role permissions, validates input, and returns JSON.
Business services	Spring service layer	Coordinates catalogue operations, order creation, status rules, delivery assignment, calculations, and notifications.
Persistence	Spring Data JPA / repositories and explicit JDBC demonstration DAO	Stores application entities and demonstrates PreparedStatement CRUD for the required Java rubric.
Database	MySQL	Stores users, catalogue data, carts, orders, deliveries, audit history, and activity records.
Notifications	Firebase Cloud Messaging	Sends new-order, delivery-assignment, and customer status notifications to registered devices.
File storage	Persistent image storage plus image URL/key	Keeps uploaded product images available after restarts and deployment.

3.3 Modules

1. Registration and Authentication: Allows customers and staff to sign in, maintains password hashes, and applies role-based access.

2. Product and Category Management: Allows the Owner to create, edit, deactivate, and organize products and categories, including English and Arabic fields.

3. Product Image Management: Accepts a validated image URL or an image uploaded from device storage and stores a persistent reference.

4. Cart and Preferences: Maintains cart items and quantities, shows feedback after adding an item, and saves the customer's preferences.

5. Profile and Addresses: Supports editable profile information, password changes, saved addresses, and checkout address selection.

6. Order Management: Creates order records, preserves item-price/name snapshots, and manages the allowed order-status transitions.

7. Supervisor Operations: Receives new-order notifications, changes an order to Processing or Cancelled, assigns Delivery users, and selects dispatch quantities.

8. Delivery Operations: Shows assigned deliveries, records acceptance, tracks dispatched items, and marks delivery completion.

9. Statistics and Audit: Calculates order/product summaries and records status changes and relevant staff actions with timestamps.

10. Notifications: Registers FCM device tokens and notifies the relevant roles when orders or delivery assignments require attention.

11. Checkout and Payment Readiness: Supports COD as the available payment method; online gateway fields remain future-ready but inactive.

12. Database and Reporting: Stores data consistently and supports queries, functions, procedures, triggers, and report summaries.

4. Functional Requirements

The following requirements define the Nabta system. The implementation and evidence placeholders in Part 2 must be checked before the report is submitted.

ID	Requirement	Description
FR-01	Authentication	The system shall allow registered users to log in securely and shall store only password hashes.
FR-02	Roles	The system shall support Customer, Owner/Admin, Supervisor, and Delivery roles with separate permissions.
FR-03	Role assignment	Only the Owner shall be allowed to promote an eligible user to Supervisor or Delivery.
FR-04	Profile management	A user shall be able to update permitted profile fields and change a password after appropriate verification.
FR-05	Phone validation	A mobile-number field shall contain digits only and meet the configured minimum length (at least seven digits).
FR-06	Bilingual catalogue	Product and category names shall store English and Arabic values and display both, with their order adapted to the selected language.
FR-07	Catalogue administration	The Owner shall add, edit, deactivate, and manage products and categories.
FR-08	Product images	The Owner shall specify an image URL or upload a supported image from device storage.
FR-09	Product browsing	Customers shall view products and categories, search the catalogue, and open product details.
FR-10	Cart	Customers shall add items, receive an added-to-cart confirmation, and increase or decrease item quantities.
FR-11	Preferences	Customers shall add/remove products from preferences on the catalogue and product-detail screens.
FR-12	Addresses	Customers shall save multiple delivery addresses and select a saved address during checkout.
FR-13	COD checkout	The current checkout shall support Cash on Delivery. Online-payment options shall remain disabled until a provider is integrated.
FR-14	Order creation	The backend shall validate current products, prices, quantities, stock, delivery details, and totals before creating an order.
FR-15	Supervisor processing	A Supervisor shall be able to process or cancel an eligible order and assign a Delivery user.
FR-16	Partial dispatch	A Supervisor shall be able to dispatch all remaining items or choose specific items and quantities for a delivery assignment.
FR-17	Delivery acceptance	The assigned Delivery user shall receive a notification and confirm acceptance in the app.
FR-18	Delivery completion	The assigned Delivery user shall mark the assignment delivered/completed. The overall order shall be derived from its item and delivery states.
FR-19	Partial delivery status	When some items are delivered and the remaining quantities are cancelled, the Owner shall see Partially Delivered, with remaining quantities and amounts accounted for.
FR-20	Audit trail	The system shall record order-status changes, the responsible user, assignment events, and timestamps.
FR-21	Employee activity	The Owner shall be able to review relevant employee actions within the application.
FR-22	Statistics	The Owner dashboard shall summarize cancelled, delivered, partially delivered, and other orders, along with product statistics.
FR-23	Notifications	New orders shall notify the Owner and registered Supervisors; delivery assignments shall notify the assigned Delivery user; relevant status changes shall notify the customer.
FR-24	Saudi currency	Prices and order totals shall be displayed in SAR. Currency display shall not be confused with automatic currency conversion.
FR-25	App sharing and branding	The Android app shall use Nabta branding and provide an About Us route for sharing/downloading the release APK when publicly hosted.
FR-26	Bilingual layout	English and Arabic screens shall preserve readability and appropriate LTR/RTL direction.

4.1 Non-Functional Requirements

Requirement	Description
Security	Authentication, role-based authorization, password hashing, input validation, least privilege, and no secrets in client code.
Integrity	Primary/foreign keys, appropriate constraints, transactional order operations, and validation of stock and quantities.
Usability	Clear feedback, readable controls, mobile-responsive design, accessible contrast, and Arabic RTL support.
Reliability	Useful error messages, controlled retries, persistent image storage, and idempotent handling of duplicate notification/webhook events.
Maintainability	Layered Java architecture, reusable frontend functions, clear database naming, and environment-specific configuration.
Performance	Indexes on frequent lookup keys and efficient queries for catalogue, customer orders, assignments, and dashboard summaries.
Portability	Android packaging through Capacitor and deployable Java backend with externalized configuration.

Requirement	Description
Privacy	Minimum necessary personal data, controlled access to customer information, and careful handling of logs and screenshots.

5. Entities, Relationships and Attributes

This section describes the conceptual data model. Entity names correspond to the planned relational schema; the exact table definitions should be reconciled with the final MySQL implementation.

Entity	Purpose	Main attributes (PK/FK identified)
USER	Stores customer and staff accounts.	user_id (PK), full_name, email, mobile_number, password_hash, role, is_active, created_at.
ADDRESS	Stores a saved delivery address for a user.	address_id (PK), user_id (FK), recipient_name, mobile_number, address lines, city, postal_code, is_default.
DEVICE_TOKEN	Stores FCM device registration for notifications.	device_token_id (PK), user_id (FK), token (unique), platform, enabled, last_registered_at.
CATEGORY	Organizes products and stores bilingual labels.	category_id (PK), name_en, name_ar, description_en, description_ar, is_active.
PRODUCT	Stores bilingual catalogue data and inventory.	product_id (PK), name_en, name_ar, description_en, description_ar, price_sar, stock_quantity, image_url, category_id (FK), is_active.
CART	Represents a customer's current cart.	cart_id (PK), user_id (FK, unique), created_at, updated_at.
CART_ITEM	Stores each product and quantity in a cart.	cart_item_id (PK), cart_id (FK), product_id (FK), quantity; unique (cart_id, product_id).
PREFERENCE	Stores a customer's saved products.	Composite PK (user_id, product_id), both FKs; created_at.
CUSTOMER_ORDER	Stores the order header, address reference, status, COD method, and totals.	order_id (PK), customer_id (FK), address_id (FK), status, payment_method, payment_status, subtotal_sar, delivery_fee_sar, total_sar, updated_by_user_id (FK), timestamps.
ORDER_ITEM	Stores products purchased and historical price/name snapshots.	order_item_id (PK), order_id (FK), product_id (FK), product name snapshots, unit_price_sar, ordered_quantity, cancelled_quantity.
PAYMENT_TRANSACTION	Stores payment records and reserves fields for future provider integration.	payment_id (PK), order_id (FK), provider_reference, amount_sar, currency, status, verified_at, created_at.
DELIVERY_ASSIGNMENT	Records the Supervisor's assignment of an order to a Delivery user.	delivery_assignment_id (PK), order_id (FK), supervisor_id (FK), delivery_user_id (FK), status, assigned_at, accepted_at, delivered_at.
DELIVERY_ASSIGNMENT_ITEM	Stores the order-line quantities sent in a delivery.	assignment_item_id (PK), delivery_assignment_id (FK), order_item_id (FK), dispatched_quantity, delivered_quantity.
ORDER_STATUS_HISTORY	Records each status change and its actor.	history_id (PK), order_id (FK), changed_by_user_id (FK), old_status, new_status, change_note, changed_at.
EMPLOYEE_ACTIVITY	Records relevant Owner/Supervisor/Delivery actions.	activity_id (PK), employee_user_id (FK), action_type, entity_type, entity_id, description, occurred_at.

5.1 Main Relationships

Relationship	Cardinality	Explanation
USER — ADDRESS	1:N	One user can save multiple delivery addresses; each address belongs to one user.
USER — DEVICE_TOKEN	1:N	One user may register multiple devices; each token belongs to one user.
CATEGORY — PRODUCT	1:N	A category contains many products; each active product belongs to one category.
USER — CART	1:0..1	A user may have one active cart; each cart belongs to one user.
CART — CART_ITEM	1:N	A cart contains one or more cart-item rows.
PRODUCT — CART_ITEM	1:N	A product can appear in many carts.
USER — PRODUCT through PREFERENCE	M:N	A user can save many products; a product can be saved by many users.
USER — CUSTOMER_ORDER	1:N	A customer can place many orders; each order belongs to one customer.
CUSTOMER_ORDER — ORDER_ITEM	1:N	An order contains its item lines.

Relationship	Cardinality	Explanation
CUSTOMER_ORDER — DELIVERY_ASSIGNMENT	1:N	An order may be fulfilled through one or more delivery assignments.
DELIVERY_ASSIGNMENT — DELIVERY_ASSIGNMENT_ITEM	1:N	Each assignment specifies the quantities sent for selected order-item rows.
CUSTOMER_ORDER — ORDER_STATUS_HISTORY	1:N	A status history records each meaningful order status change.
USER — EMPLOYEE_ACTIVITY	1:N	A staff member can perform many logged actions.
CUSTOMER_ORDER — PAYMENT_TRANSACTION	1:N	An order may have one or more payment attempts or payment records; COD remains current.

6. ER and EER Diagrams

The ER diagrams use the traditional Chen notation required for the database lab: rectangles represent entity sets, ovals represent attributes, diamonds represent relationship sets, underlined attributes identify keys, and 1/N/M labels show relationship cardinality. The EER diagrams extend the conceptual model by showing the user-role specialization.

6.1 ER Diagram — Part 1

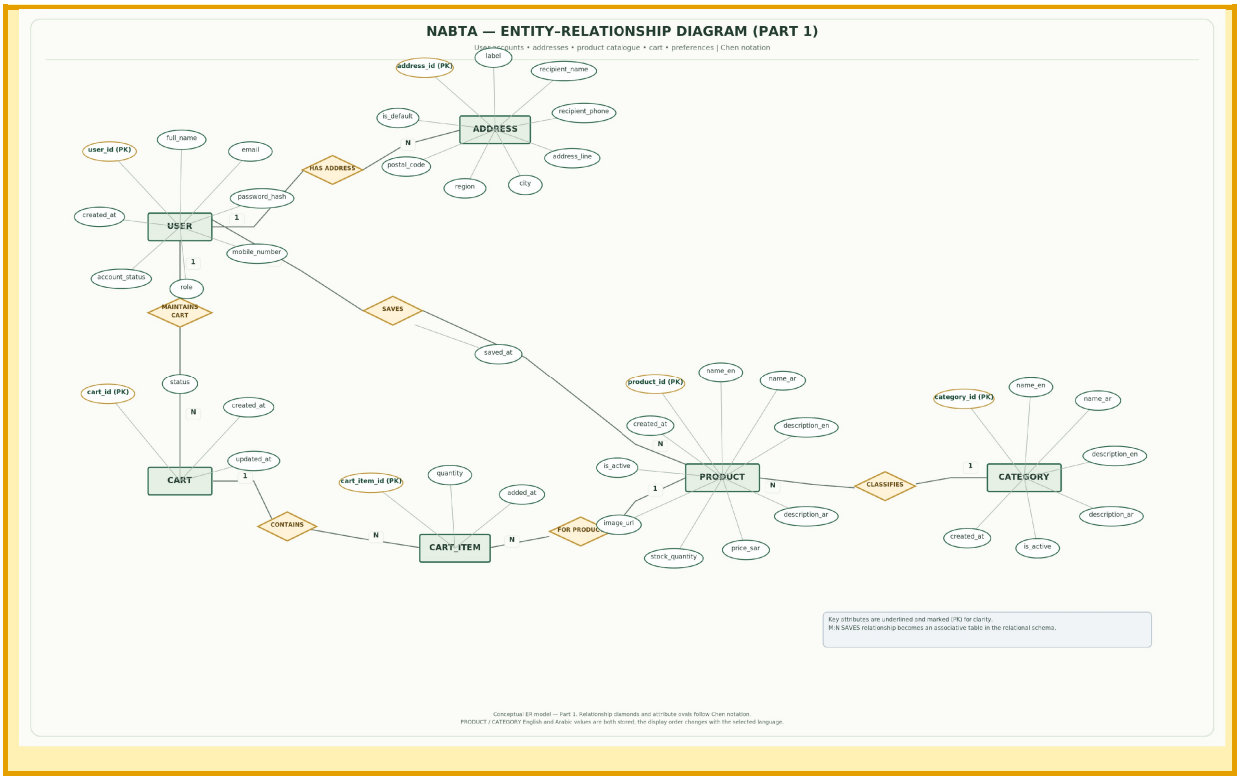


Figure 2. ER diagram — accounts, catalogue, cart, and preferences.

6.2 ER Diagram — Part 2

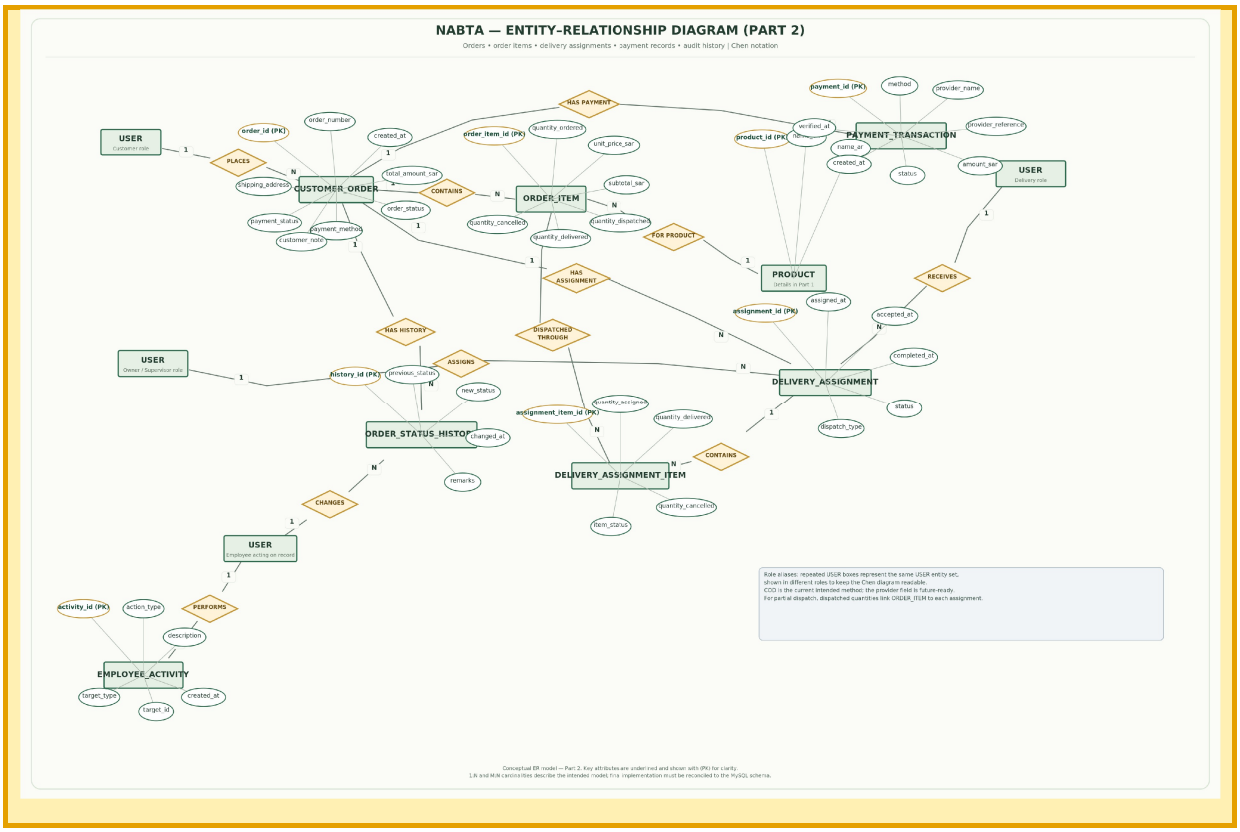


Figure 3. ER diagram — orders, delivery, and activity history.

6.3 EER Diagram — Part 1

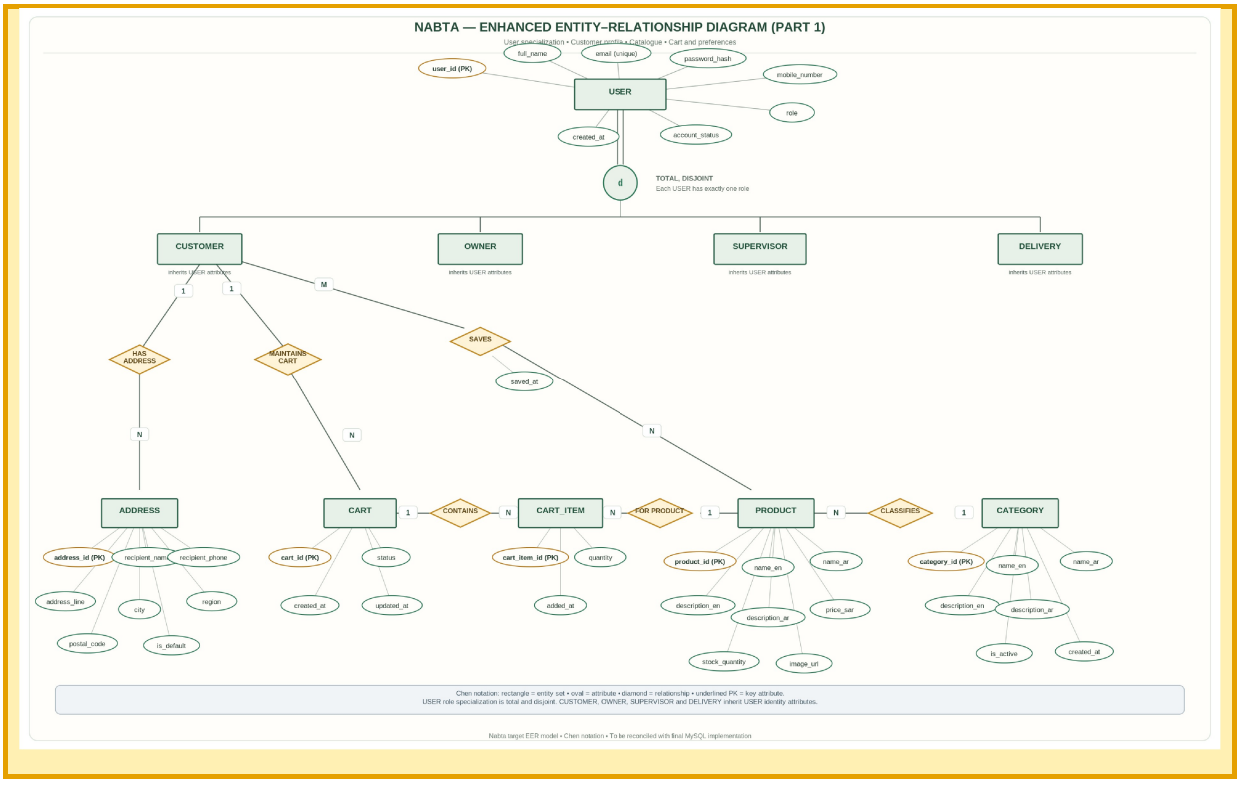


Figure 4. EER diagram — user specialization, catalogue, and customer features.

6.4 EER Diagram — Part 2

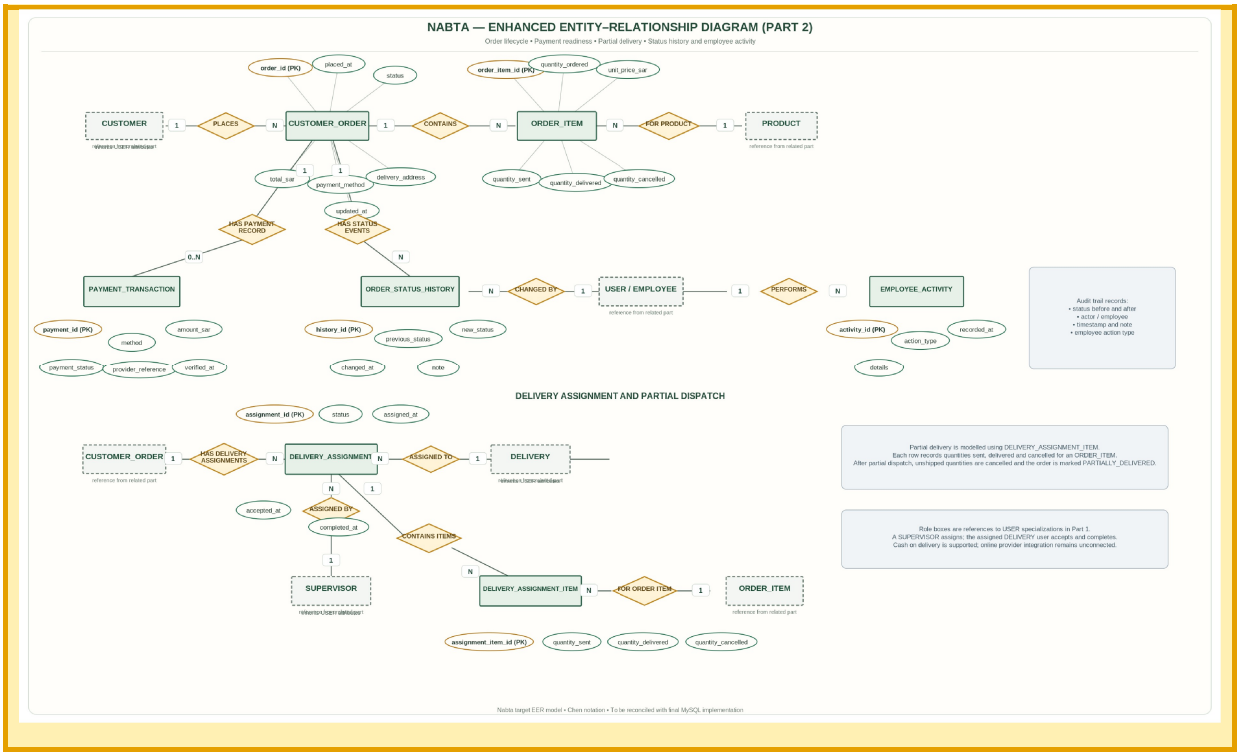


Figure 5. EER diagram — order processing, delivery, and audit records.

6.5 Role Specialization Constraints

The model uses USER as a common superclass with role specializations for Customer, Owner/Admin, Supervisor, and Delivery. The specializations are disjoint in the normal account model: one account has one active primary role. In relational implementation, this can be represented by a role discriminator column in USER, while authorization rules determine which operations each role can perform. If the project permits multiple roles per account in the future, the database should change to an explicit user-role bridge table rather than silently assigning multiple values to the role column.

NABTA PROJECT REPORT — CONTINUATION

7. Relational Schema

The relational schema maps the conceptual entities to MySQL relations. Each table has a primary key to identify records and foreign keys to preserve referential integrity. The schema keeps bilingual catalogue values in separate English and Arabic columns and represents prices with DECIMAL values in SAR. The final database must retain historical product names and prices for order records even when catalogue items are later edited.

7.1 Relational Schema Diagram — Part 1

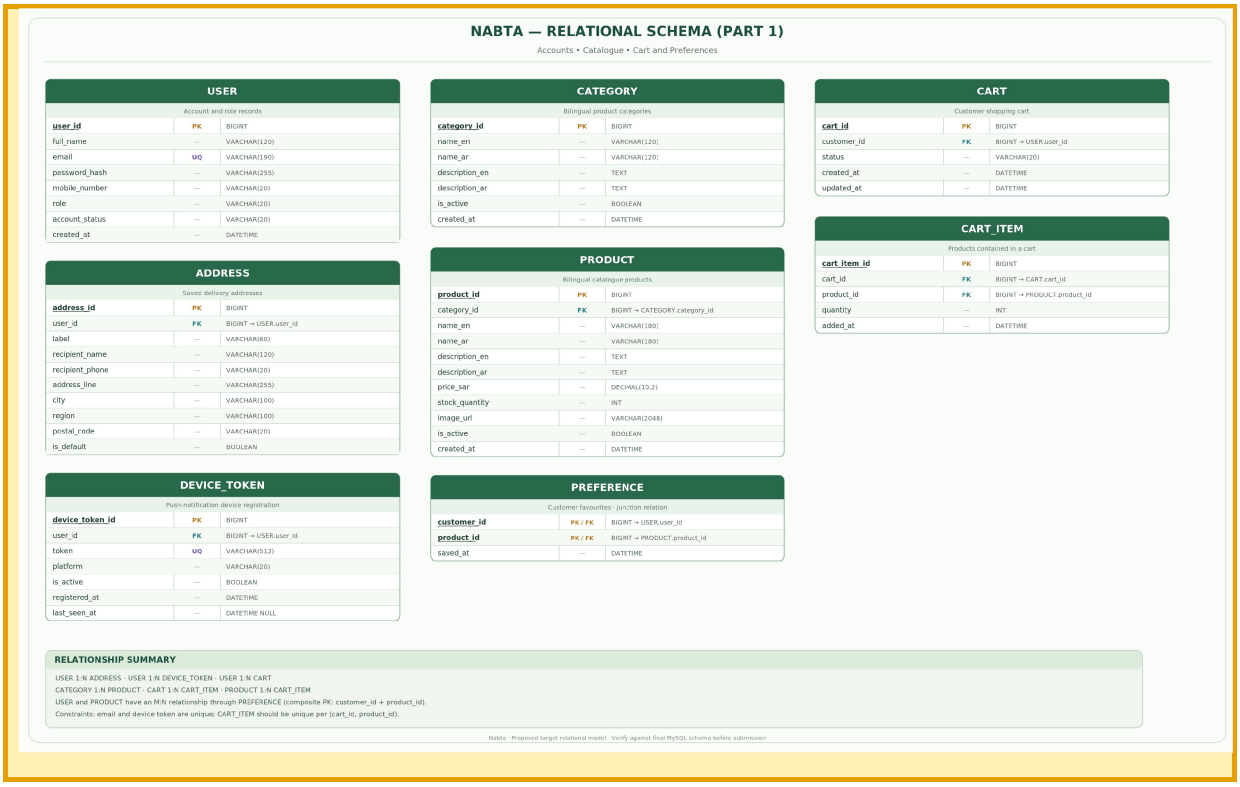


Figure 6. Relational schema — accounts, catalogue, cart, and preferences.

7.2 Relational Schema Diagram — Part 2

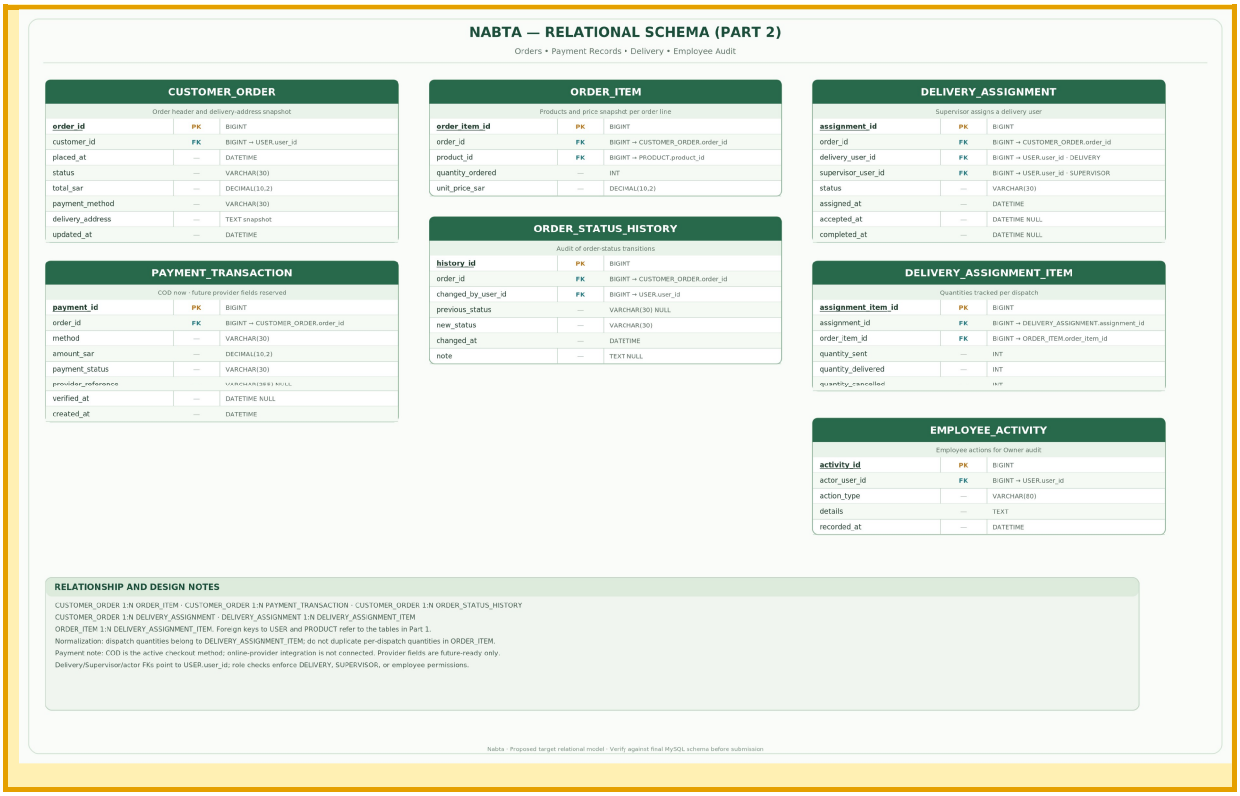


Figure 7. Relational schema — orders, delivery, and employee activity.

Relation	Primary key	Non-key attributes (summary)	Key constraints
USER	user_id	full_name, email, mobile_number, password_hash, role, is_active, created_at	PK: user_id; email unique.
ADDRESS	address_id	user_id, recipient_name, mobile_number, address_line1, address_line2, city, postal_code, is_default	FK user_id → USER.user_id.
DEVICE_TOKEN	device_token_id	user_id, token, platform, notification_enabled, last_registered_at	FK user_id → USER.user_id; token unique.
CATEGORY	category_id	name_en, name_ar, description_en, description_ar, is_active	Bilingual text stored in separate columns.
PRODUCT	product_id	name_en, name_ar, description_en, description_ar, price_sar, stock_quantity, image_url, category_id, is_active	FK category_id → CATEGORY.category_id.
CART	cart_id	user_id, created_at, updated_at	FK user_id → USER.user_id; normally one active cart per user.
CART_ITEM	cart_item_id	cart_id, product_id, quantity	FKs to CART and PRODUCT; unique (cart_id, product_id).
PREFERENCE	user_id + product_id	created_at	Composite PK (user_id, product_id); both fields are FKs.
CUSTOMER_ORDER	order_id	customer_id, address_id, status, payment_method, payment_status, subtotal_sar, delivery_fee_sar, total_sar, updated_by_user_id, timestamps	FKs to USER and ADDRESS; status changes must be audited.
ORDER_ITEM	order_item_id	order_id, product_id, name snapshots, unit_price_sar, ordered_quantity, cancelled_quantity	FKs to order and product; preserves historical line details.
PAYMENT_TRANSACTION	payment_id	order_id, provider_reference, amount_sar, currency, status, verified_at, created_at	FK order_id → CUSTOMER_ORDER.order_id; online provider currently disconnected.
DELIVERY_ASSIGNMENT	delivery_assignment_id	order_id, supervisor_id, delivery_user_id, status, assigned_at, accepted_at, delivered_at	FKs to CUSTOMER_ORDER and USER.
DELIVERY_ASSIGNMENT_ITEM	assignment_item_id	delivery_assignment_id, order_item_id, dispatched_quantity, delivered_quantity	FKs to assignment and order item.

Relation	Primary key	Non-key attributes (summary)	Key constraints
ORDER_STATUS_HISTORY	history_id	order_id, changed_by_user_id, old_status, new_status, change_note, changed_at	FKs to CUSTOMER_ORDER and USER.
EMPLOYEE_ACTIVITY	activity_id	employee_user_id, action_type, entity_type, entity_id, description, occurred_at	FK employee_user_id → USER.user_id.

7.3 Key and Referential Integrity Rules

- A primary-key value must be unique and non-null.
- Every foreign-key value must refer to an existing parent record, except where an optional relationship explicitly permits NULL.
- Product price values use DECIMAL(12,2), not floating-point types. Quantities use integer types and must not be negative.
- Email and device-token values should be unique where the application requires a single account/token identity.
- Product records referenced by historical order items should normally be deactivated rather than physically deleted.
- Passwords are stored as one-way hashes; plain-text passwords must never be stored in the database.
- Delivery item quantities and cancelled quantities must be checked against the ordered quantity inside a transaction.

8. Application of Codd's Rules

The following assessment follows the twelve-rule presentation used in the supplied sample report. "Supported" means the design provides the relevant capability; it does not mean that every rule is automatically satisfied regardless of configuration or implementation.

Rule	Assessment	Application to Nabta
1. Information Rule	Supported	Application data is represented as values in relational tables and rows.
2. Guaranteed Access Rule	Supported	A data item can be addressed through its table, primary key, and column, subject to access permissions.
3. Systematic Treatment of NULL Values	Supported with constraints	NULL is used only for unknown, unavailable, or inapplicable values. Required attributes are marked NOT NULL.
4. Active Online Catalog	Supported by MySQL	Schema metadata is available through MySQL catalog tables such as INFORMATION_SCHEMA.
5. Comprehensive Data Sublanguage	Supported	SQL provides DDL, DML, query, privilege, and transaction-control operations.
6. View Updating Rule	Partially supported	Some simple MySQL views are updatable; complex joins, aggregates, and grouped views may not be.
7. High-Level Insert, Update and Delete	Supported	SQL supports set-oriented INSERT, UPDATE, and DELETE operations.
8. Physical Data Independence	Largely supported	Physical storage changes can often be managed by MySQL without changing client screens, although performance-related tuning may be needed.
9. Logical Data Independence	Partially supported	Well-defined APIs and views reduce coupling, but changes to tables or field meanings can still require application changes.
10. Integrity Independence	Partially supported	Primary keys, foreign keys, CHECK, UNIQUE, and NOT NULL constraints are maintained in the database; some business rules also remain in Java services.
11. Distribution Independence	Limited / future deployment dependent	The current client communicates through an API, hiding database location from the user. Distributed database placement is not yet a demonstrated feature.
12. Non-Subversion Rule	Supported when access is configured correctly	The API applies role checks and database permissions limit direct access. Privileged administrators still require careful control and auditing.

9. Functional Dependencies

A functional dependency X → Y means that the value of attribute set X determines the value of attribute set Y. The dependencies below describe the relational schema and should be checked against the implemented columns and keys.

Relation	Functional dependency
USER	user_id → full_name, email, mobile_number, password_hash, role, is_active, created_at; email → user_id (unique email).
ADDRESS	address_id → user_id, recipient_name, mobile_number, address_line1, address_line2, city, postal_code, is_default.
DEVICE_TOKEN	device_token_id → user_id, token, platform, notification_enabled, last_registered_at; token → device_token_id if globally unique.
CATEGORY	category_id → name_en, name_ar, description_en, description_ar, is_active.
PRODUCT	product_id → name_en, name_ar, description_en, description_ar, price_sar, stock_quantity, image_url, category_id, is_active.
CART	cart_id → user_id, created_at, updated_at; if one cart per user is enforced, user_id → cart_id.
CART_ITEM	cart_item_id → cart_id, product_id, quantity; if cart_id/product_id is unique, (cart_id, product_id) → quantity.
PREFERENCE	(user_id, product_id) → created_at.
CUSTOMER_ORDER	order_id → customer_id, address_id, status, payment_method, payment_status, subtotal_sar, delivery_fee_sar, total_sar, updated_by_user_id, timestamps.
ORDER_ITEM	order_item_id → order_id, product_id, name snapshots, unit_price_sar, ordered_quantity, cancelled_quantity.
PAYMENT_TRANSACTION	payment_id → order_id, provider_reference, amount_sar, currency, status, verified_at, created_at.
DELIVERY_ASSIGNMENT	delivery_assignment_id → order_id, supervisor_id, delivery_user_id, status, assigned_at, accepted_at, delivered_at.
DELIVERY_ASSIGNMENT_ITEM	assignment_item_id → delivery_assignment_id, order_item_id, dispatched_quantity, delivered_quantity.
ORDER_STATUS_HISTORY	history_id → order_id, changed_by_user_id, old_status, new_status, change_note, changed_at.
EMPLOYEE_ACTIVITY	activity_id → employee_user_id, action_type, entity_type, entity_id, description, occurred_at.

9.1 Candidate Keys

Most relations use a single-column primary key such as product_id or order_id. PREFERENCE uses the composite key (user_id, product_id), because a customer should save a given product at most once. CART_ITEM can additionally enforce UNIQUE(cart_id, product_id) so that each product appears once in a cart and its quantity is updated rather than duplicated. Any alternate unique key, such as email or device token, should be enforced with a UNIQUE constraint if the business rule requires it.

10. Anomaly Analysis

Anomalies arise when unrelated facts are stored redundantly in one relation. A normalized schema separates user, catalogue, order, and delivery information so one fact is maintained in one appropriate location. Historical snapshots are an intentional exception: they preserve what the customer ordered at the time of purchase.

Anomaly	Example risk	Design prevention
Update anomaly	If a category name were repeated in every product row, renaming the category could require changing many rows and might leave inconsistent values.	Keep category names in CATEGORY; PRODUCT stores category_id only.
Insertion anomaly	If category information existed only inside PRODUCT, the business could not create a category until a product existed for it.	Store CATEGORY independently of PRODUCT.
Deletion anomaly	Deleting the last product in a category could accidentally remove the only copy of the category name if both facts were in one table.	Separate CATEGORY and PRODUCT; prefer deactivation for referenced products.

Anomaly	Example risk	Design prevention
Order-history anomaly	If order items referenced only the current product name and price, later catalogue edits could change the apparent historical receipt.	Store product-name snapshots and unit_price_sar in ORDER_ITEM.
Delivery-quantity anomaly	If each partial dispatch overwrote the order's original quantity, the system could lose the original order amount and previous dispatch history.	Keep the ordered quantity in ORDER_ITEM and store each dispatch/delivery quantity in DELIVERY_ASSIGNMENT_ITEM.

11. Normalization

Normalization organizes attributes and relations to reduce unnecessary duplication and protect consistency. The data model targets Third Normal Form (3NF) for core catalogue, account, order, delivery, and audit entities, while preserving historical order snapshots where needed for business correctness.

11.1 First Normal Form (1NF)

A relation is in 1NF when each attribute contains one atomic value and there are no repeating groups. Nabta stores one product name per language column, one device token per DEVICE_TOKEN row, one address per ADDRESS row, and one product line per ORDER_ITEM row. A list of product IDs or multiple addresses should not be stored as comma-separated text in a single field.

11.2 Second Normal Form (2NF)

A relation is in 2NF when it is in 1NF and each non-key attribute depends on the whole candidate key, not just part of a composite key. PREFERENCE uses (user_id, product_id) as its composite key; its created_at value describes that specific user-product preference pair. Attributes such as a user email belong in USER, not in PREFERENCE, because email depends on user_id alone.

11.3 Third Normal Form (3NF)

A relation is in 3NF when it is in 2NF and non-key attributes do not depend transitively on the key through another non-key attribute. PRODUCT stores category_id but not a repeated category name; the category names remain in CATEGORY. ORDER_STATUS_HISTORY stores the actor's user ID and status-change fields, while user profile information remains in USER. This separation avoids copying mutable profile or category data into many unrelated rows.

11.4 Historical Snapshot Note

ORDER_ITEM stores the product name and unit price as they were at order time. This is intentional historical data, not a replacement for the normalized PRODUCT table. If the Owner later changes a product name or price, the prior order still needs to show the values agreed to when it was placed. The same principle applies to the delivery address used for an order if the customer edits the saved address afterward.

11.5 Normalization Summary

Normal form	Rule	Nabta application
1NF	Atomic values; no repeating groups.	Addresses, device tokens, cart items, order items, and delivery-item quantities are represented as separate rows.
2NF	No partial dependency on part of a composite key.	PREFERENCE attributes depend on the whole (user_id, product_id) key.
3NF	No non-key attribute depends transitively on the primary key.	Category details stay in CATEGORY; user details stay in USER; status changes stay in ORDER_STATUS_HISTORY.

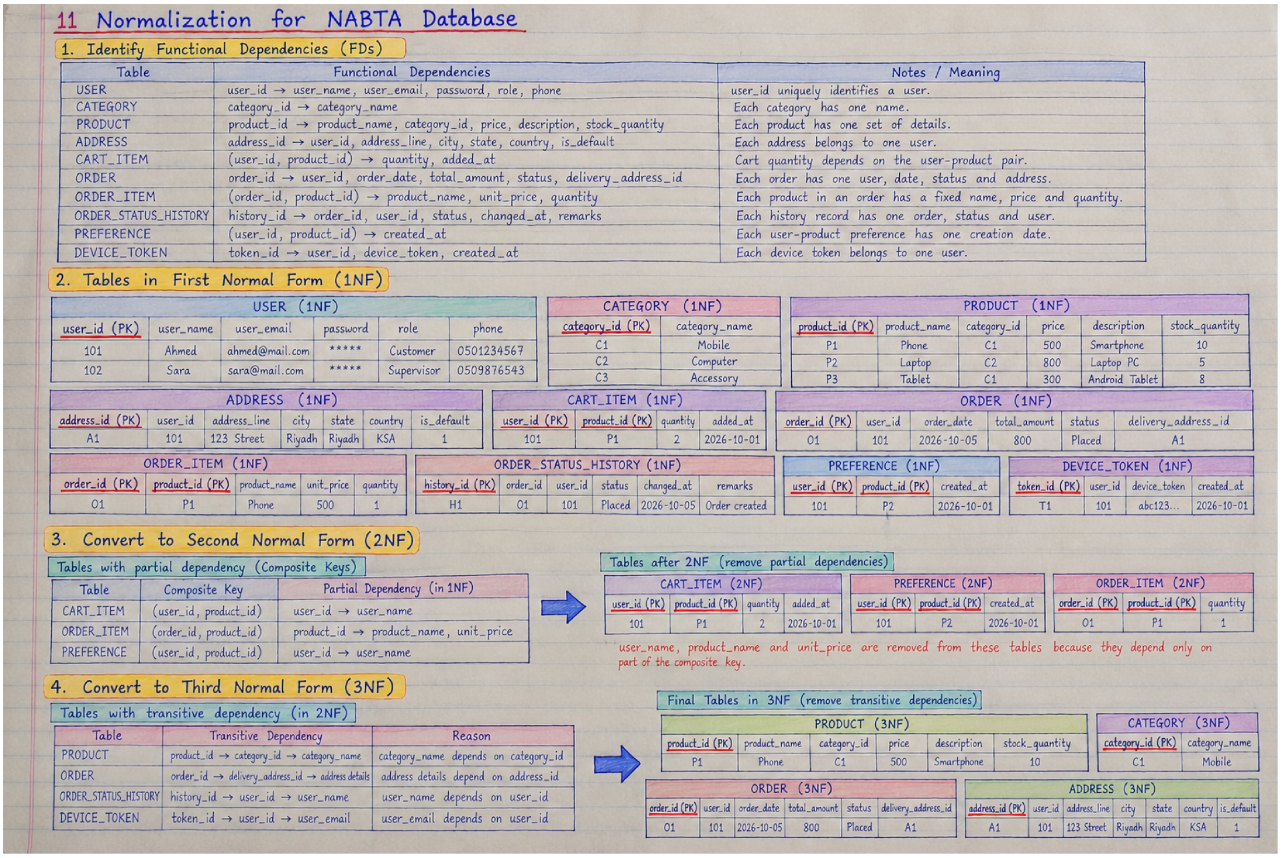


Table	Functional Dependencies	Notes / Meaning
USER	user_id → user_name, user_email, password, role, phone	user_id uniquely identifies a user.
CATEGORY	category_id → category_name	Each category has one name.
PRODUCT	product_id → product_name, category_id, price, description, stock_quantity	Each product has one set of details.
ADDRESS	address_id → user_id, address_line, city, state, country, is_default	Each address belongs to one user.
CART_ITEM	(user_id, product_id) → quantity, added_at	Cart quantity depends on the user-product pair.
ORDER	order_id → user_id, order_date, total_amount, status, delivery_address_id	Each order has one user, date, status and address.
ORDER_ITEM	(order_id, product_id) → product_name, unit_price, quantity	Each product in an order has a fixed name, price and quantity.
ORDER_STATUS_HISTORY	history_id → order_id, user_id, status, changed_at, remarks	Each history record has one order, status and user.
PREFERENCE	(user_id, product_id) → created_at	Each user-product preference has one creation date.
DEVICE_TOKEN	token_id → user_id, device_token, created_at	Each device token belongs to one user.

USER (1NF)

user_id (PK)	user_name	user_email	password	role	phone
101	Ahmed	ahmed@mail.com	*****	Customer	0501234567
102	Sara	sara@mail.com	*****	Supervisor	0509876543

CATEGORY (1NF)

category_id (PK)	category_name
C1	Mobile
C2	Computer
C3	Accessory

PRODUCT (1NF)

product_id (PK)	product_name	category_id	price	description	stock_quantity
P1	Phone	C1	500	Smartphone	10
P2	Laptop	C2	800	Laptop PC	5
P3	Tablet	C1	300	Android Tablet	8

ADDRESS (1NF)

address_id (PK)	user_id	address_line	city	state	country	is_default
A1	101	123 Street	Riyadh	Riyadh	KSA	1

CART_ITEM (1NF)

user_id (PK)	product_id (PK)	quantity	added_at
101	P1	2	2026-10-01

ORDER (1NF)

order_id (PK)	user_id	order_date	total_amount	status	delivery_address_id
O1	101	2026-10-05	800	Placed	A1

ORDER_ITEM (1NF)

order_id (PK)	product_id (PK)	product_name	unit_price	quantity
O1	P1	Phone	500	1

ORDER_STATUS_HISTORY (1NF)

history_id (PK)	order_id	user_id	status	changed_at	remarks
H1	O1	101	Placed	2026-10-05	Order created

PREFERENCE (1NF)

user_id (PK)	product_id (PK)	created_at
101	P2	2026-10-01

DEVICE_TOKEN (1NF)

token_id (PK)	user_id	device_token	created_at
T1	101	abc123...	2026-10-01

Table	Composite Key	Partial Dependency (in 1NF)
CART_ITEM	(user_id, product_id)	user_id → user_name
ORDER_ITEM	(order_id, product_id)	product_id → product_name, unit_price
PREFERENCE	(user_id, product_id)	user_id → user_name

CART_ITEM (2NF)

user_id (PK)	product_id (PK)	quantity	added_at
101	P1	2	2026-10-01

PREFERENCE (2NF)

user_id (PK)	product_id (PK)	created_at
101	P2	2026-10-01

ORDER_ITEM (2NF)

order_id (PK)	product_id (PK)	quantity
O1	P1	1

Table	Transitive Dependency	Reason
PRODUCT	product_id → category_id → category_name	category_name depends on category_id
ORDER	order_id → delivery_address_id → address details	address details depend on address_id
ORDER_STATUS_HISTORY	history_id → user_id → user_name	user_name depends on user_id
DEVICE_TOKEN	token_id → user_id → user_email	user_email depends on user_id

PRODUCT (3NF)

product_id (PK)	product_name	category_id	price	description	stock_quantity
P1	Phone	C1	500	Smartphone	10

CATEGORY (3NF)

category_id (PK)	category_name
C1	Mobile

ORDER (3NF)

order_id (PK)	user_id	order_date	total_amount	status	delivery_address_id
O1	101	2026-10-05	800	Placed	A1

ADDRESS (3NF)

address_id (PK)	user_id	address_line	city	state	country	is_default
A1	101	123 Street	Riyadh	Riyadh	KSA	1

12. MySQL Implementation

The database is implemented using MySQL. The backend connects to MySQL through its configured JDBC data source. During development, database access is confirmed through the application logs; the academic demonstration should also include screenshots of table structure and executed SQL statements.

12.1 Example Table Definitions

The following representative DDL illustrates important keys and constraints. The final executable schema must be aligned with the actual application migrations/entities before submission; it is not a replacement for the project's complete schema.

Example 1. Account table with password hash and role.

```
CREATE TABLE `USER` (
  user_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(190) NOT NULL UNIQUE,
  mobile_number VARCHAR(20) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('CUSTOMER','ADMIN','SUPERVISOR','DELIVERY') NOT NULL DEFAULT 'CUSTOMER',
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Example 2. Bilingual catalogue tables with foreign key and value constraints.

```
CREATE TABLE CATEGORY (
  category_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name_en VARCHAR(180) NOT NULL,
  name_ar VARCHAR(180) NOT NULL,
  description_en TEXT,
  description_ar TEXT,
  is_active BOOLEAN NOT NULL DEFAULT TRUE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

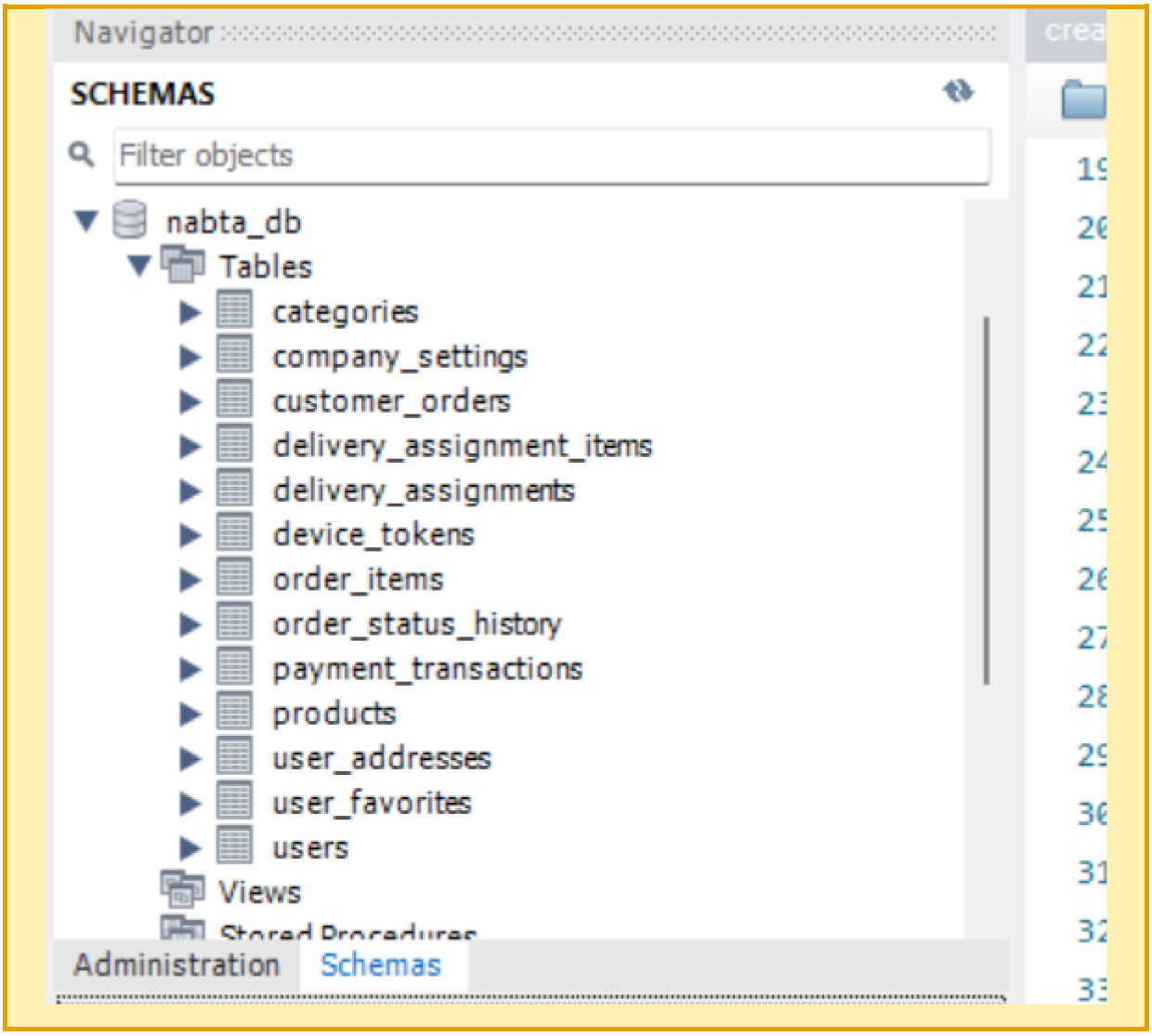
CREATE TABLE PRODUCT (
  product_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  name_en VARCHAR(180) NOT NULL,
```

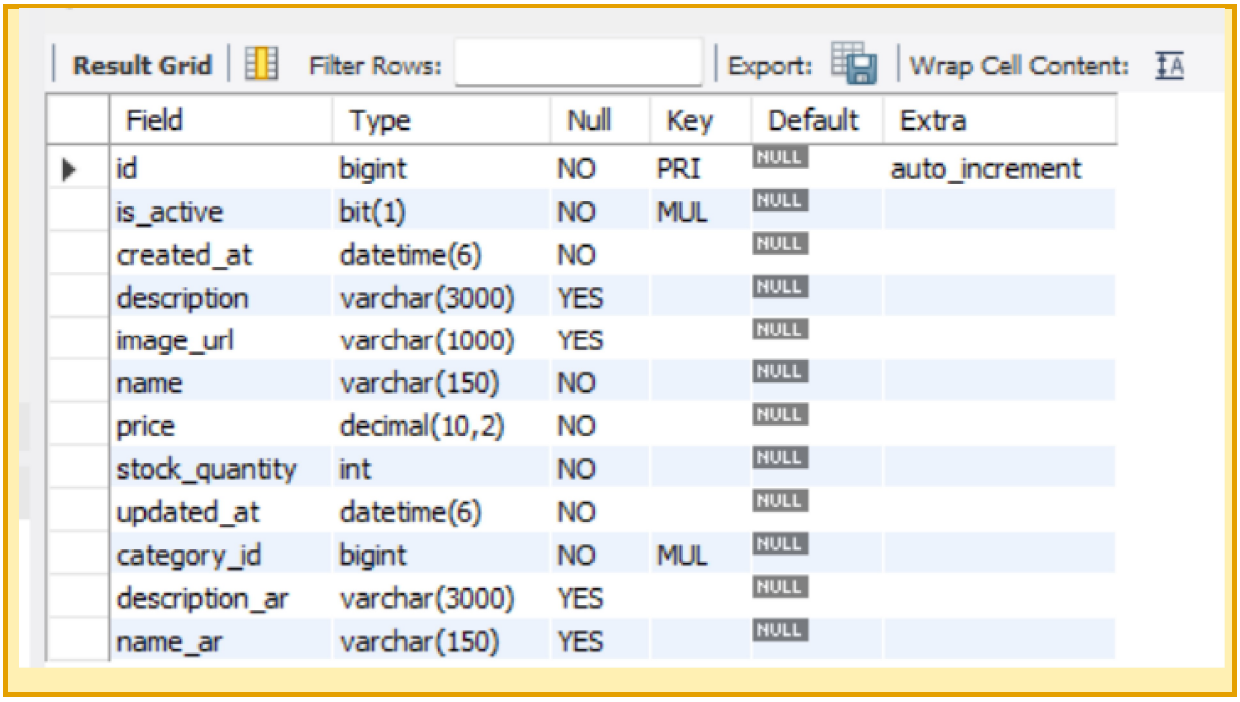
```
  name_ar VARCHAR(180) NOT NULL,
  description_en TEXT,
  description_ar TEXT,
  price_sar DECIMAL(12,2) NOT NULL CHECK (price_sar >= 0),
  stock_quantity INT NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
  image_url VARCHAR(1000),
  category_id BIGINT NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_product_category FOREIGN KEY (category_id)
    REFERENCES CATEGORY(category_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Example 3. Order header. The address reference may be supplemented by immutable delivery-address snapshot fields in the final schema.

```
CREATE TABLE CUSTOMER_ORDER (
  order_id BIGINT PRIMARY KEY AUTO_INCREMENT,
  customer_id BIGINT NOT NULL,
  address_id BIGINT NOT NULL,
  status VARCHAR(32) NOT NULL DEFAULT 'PENDING',
  payment_method VARCHAR(20) NOT NULL DEFAULT 'COD',
  payment_status VARCHAR(32) NOT NULL DEFAULT 'PENDING_COLLECTION',
  subtotal_sar DECIMAL(12,2) NOT NULL DEFAULT 0,
  delivery_fee_sar DECIMAL(12,2) NOT NULL DEFAULT 0,
  total_sar DECIMAL(12,2) NOT NULL DEFAULT 0,
  updated_by_user_id BIGINT,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_order_customer FOREIGN KEY (customer_id) REFERENCES `USER`(user_id),
  CONSTRAINT fk_order_address FOREIGN KEY (address_id) REFERENCES ADDRESS(address_id),
  CONSTRAINT fk_order_updated_by FOREIGN KEY (updated_by_user_id) REFERENCES `USER`(user_id),
  CONSTRAINT chk_order_amounts CHECK (subtotal_sar >= 0 AND delivery_fee_sar >= 0 AND total_sar >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

12.2 MySQL Screenshot Evidence





Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
is_active	bit(1)	NO	MUL	NULL	
created_at	datetime(6)	NO		NULL	
description	varchar(3000)	YES		NULL	
image_url	varchar(1000)	YES		NULL	
name	varchar(150)	NO		NULL	
price	decimal(10,2)	NO		NULL	
stock_quantity	int	NO		NULL	
updated_at	datetime(6)	NO		NULL	
category_id	bigint	NO	MUL	NULL	
description_ar	varchar(3000)	YES		NULL	
name_ar	varchar(150)	YES		NULL	

13. SQL Queries, Functions, Procedures and Triggers

This section demonstrates the types of SQL statements required by the database rubric. Run examples only against the project's intended schema or a test database. Use test orders and test products when demonstrating updates or triggers.

13.1 Catalogue Query

Query 1. Retrieve active products with both language values and both category names.

```
SELECT p.product_id, p.name_en, p.name_ar,
       p.price_sar, p.stock_quantity,
       c.name_en AS category_en, c.name_ar AS category_ar
FROM PRODUCT p
JOIN CATEGORY c ON c.category_id = p.category_id
WHERE p.is_active = TRUE
ORDER BY p.name_en;
```

The frontend can display both names simultaneously, reversing their order according to the selected language. The database stores each language independently rather than concatenating translations into a single field.

13.2 Customer Order Query

Query 2. Retrieve one customer's order history. In application code, bind the customer ID from the authenticated user, not from an untrusted client claim.

```
SELECT o.order_id, o.created_at, o.status,
       o.payment_method, o.payment_status, o.total_sar
FROM CUSTOMER_ORDER o
WHERE o.customer_id = ?
ORDER BY o.created_at DESC;
```

13.3 Supervisor Pending Orders Query

Query 3. Show orders awaiting processing. The backend must restrict access to Owner/Supervisor roles.

```
SELECT o.order_id, o.created_at, o.status, o.total_sar,
       u.full_name AS customer_name
FROM CUSTOMER_ORDER o
JOIN `USER` u ON u.user_id = o.customer_id
WHERE o.status IN ('PENDING','PROCESSING')
ORDER BY o.created_at ASC;
```

13.4 Product Statistics Query

Query 4. Summarize product quantities while preserving products with no order items.

```
SELECT p.product_id, p.name_en, p.name_ar,
       p.stock_quantity,
       COALESCE(SUM(oi.ordered_quantity - oi.cancelled_quantity), 0) AS net_ordered_qty
FROM PRODUCT p
LEFT JOIN ORDER_ITEM oi ON oi.product_id = p.product_id
GROUP BY p.product_id, p.name_en, p.name_ar, p.stock_quantity
ORDER BY net_ordered_qty DESC;
```

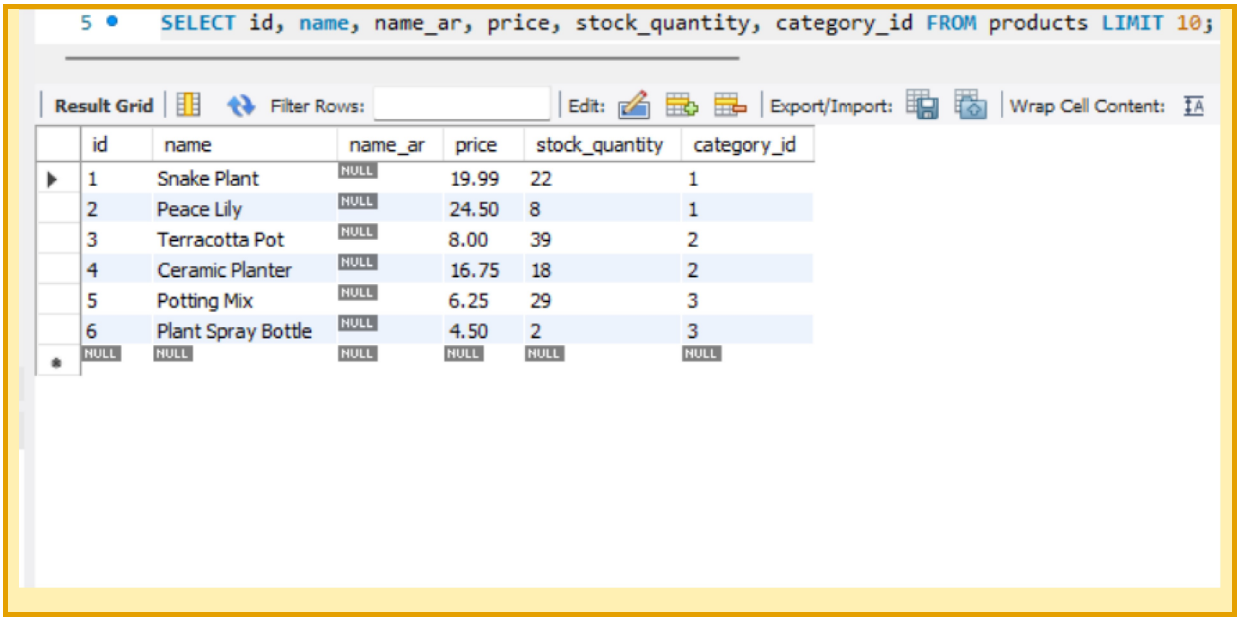
13.5 Stored Function

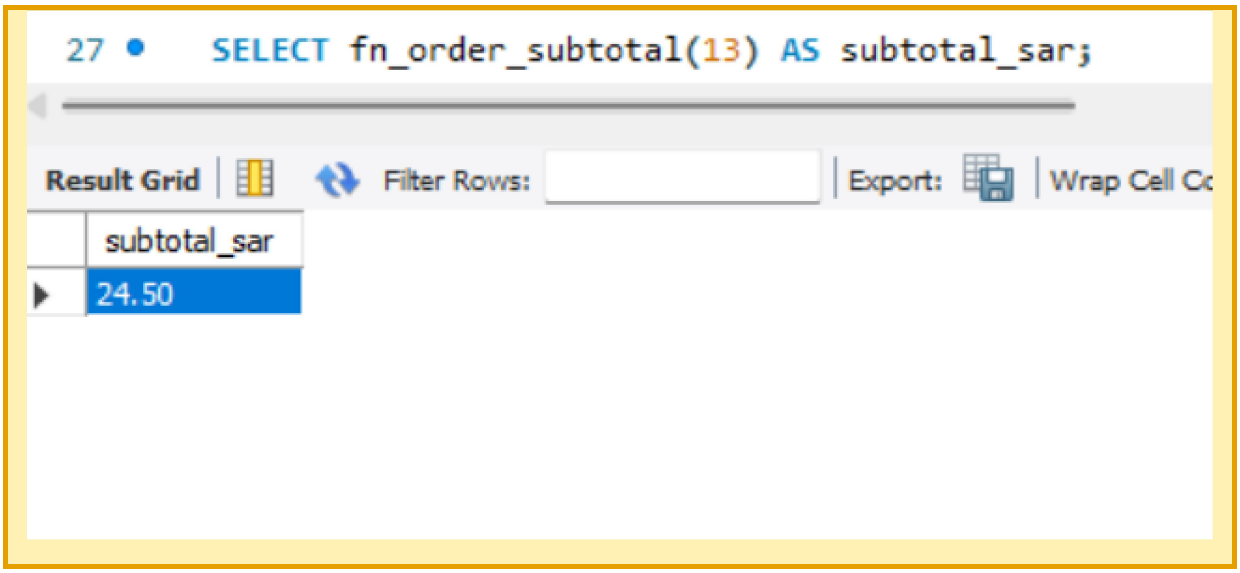
Example function. Use a valid test order ID and confirm the cancelled-quantity rule matches the final application policy.

```
DELIMITER //
CREATE FUNCTION fn_order_subtotal(p_order_id BIGINT)
RETURNS DECIMAL(12,2)
NOT DETERMINISTIC
READS SQL DATA
BEGIN
  DECLARE v_total DECIMAL(12,2) DEFAULT 0.00;
```

```
  SELECT COALESCE(SUM((ordered_quantity - cancelled_quantity) * unit_price_sar), 0.00)
    INTO v_total
  FROM ORDER_ITEM
  WHERE order_id = p_order_id;
  RETURN v_total;
END//
DELIMITER ;

-- Example call using a test order:
SELECT fn_order_subtotal(1001);
```



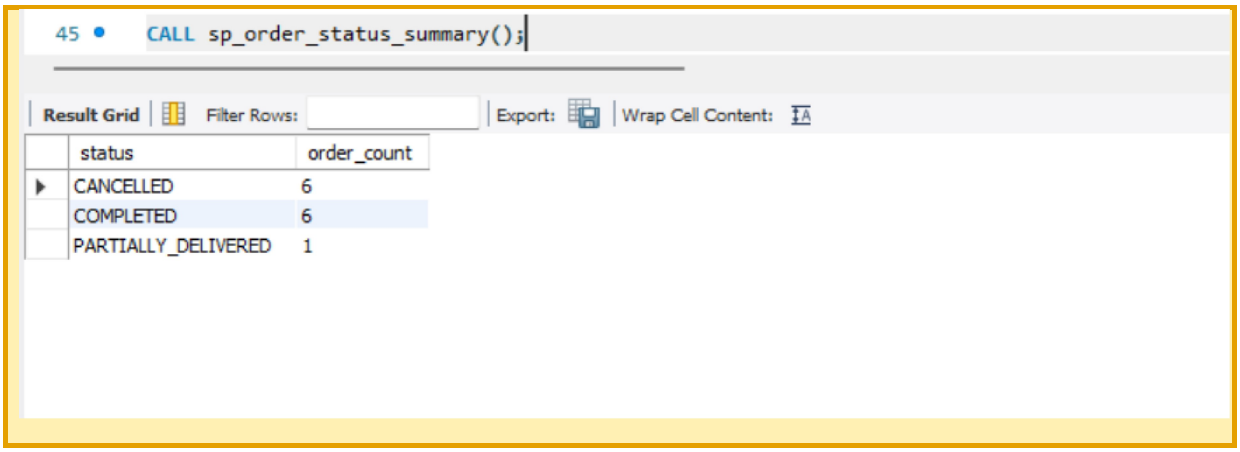


13.6 Stored Procedure

Example stored procedure. The report counts each order status for the Owner dashboard.

```
DELIMITER //
CREATE PROCEDURE sp_order_status_summary()
BEGIN
  SELECT status, COUNT(*) AS order_count
  FROM CUSTOMER_ORDER
  GROUP BY status
  ORDER BY status;
END//
DELIMITER ;
```

```
-- Example call:
CALL sp_order_status_summary();
```

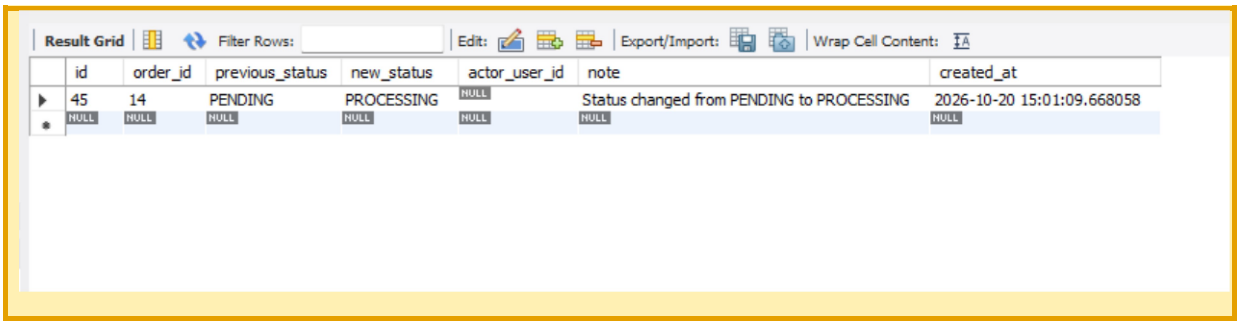


13.7 Trigger for Order-Status History

Example trigger. Confirm that updated_by_user_id is set by the backend before an order status is changed.

```
DELIMITER //
CREATE TRIGGER trg_order_status_history_after_update
AFTER UPDATE ON CUSTOMER_ORDER
FOR EACH ROW
BEGIN
  IF NOT (OLD.status <=> NEW.status) THEN
    INSERT INTO ORDER_STATUS_HISTORY
      (order_id, changed_by_user_id, old_status, new_status, change_note, changed_at)
    VALUES
      (NEW.order_id, NEW.updated_by_user_id, OLD.status, NEW.status,
       'Status changed', CURRENT_TIMESTAMP);
  END IF;
END//
DELIMITER ;
```

A trigger is database code, so it must be coordinated with any JPA lifecycle code or service that also writes history to avoid duplicate audit entries. The final application should have one clearly defined source of truth for each audit event.



14. Application of Java Concepts

This section is included to align the report with the supplied Java mini-project rubric. The final source code should contain a clear, working example of each concept and the screenshots in this section should be captured from that final source. The snippets below are explanatory examples and must be reconciled with the exact class names used in the final code.

14.1 Inheritance — 1 Mark

Inheritance allows a subclass to reuse and extend the behavior of a superclass. In Nabta's conceptual staff workflow, common employee identity can be separated from role-specific actions.

Illustrative inheritance example: Supervisor extends StaffMember.

```
abstract class StaffMember {
    protected final long userId;
    protected final String name;

    protected StaffMember(long userId, String name) {
        this.userId = userId;
        this.name = name;
    }

    public abstract void performOrderAction(long orderId);
}

class Supervisor extends StaffMember {
    public Supervisor(long userId, String name) {
        super(userId, name);
    }

    @Override
    public void performOrderAction(long orderId) {
        System.out.println("Supervisor processes order " + orderId);
    }
}
```

14.2 Collections — 1 Mark

Java collections store groups of objects efficiently. List can hold ordered search results, while Map can provide quick lookup by product ID. Collections should not replace database constraints or the database as the authoritative store.

Illustrative List and Map usage for catalogue results.

```
List<ProductDto> products = new ArrayList<>();
Map<Long, ProductDto> productsById = new HashMap<>();

for (ProductDto product : products) {
    productsById.put(product.getProductId(), product);
}
```

14.3 Exception Handling — 1 Mark

Exception handling provides controlled error responses and prevents raw database errors or stack traces from being shown to the customer. Validation errors, missing records, and database failures should be logged appropriately and mapped to safe responses.

Illustrative exception-handling pattern; exception class names may vary by project.

```
try {
    orderService.placeCodOrder(request, authenticatedUserId);
} catch (IllegalArgumentException ex) {
    throw new BadRequestException("Invalid order details", ex);
} catch (DataAccessException ex) {
    // Log a safe error identifier; do not expose database credentials or SQL.
    throw new ServiceUnavailableException("Order could not be saved", ex);
}
```

14.4 JDBC CRUD with PreparedStatement — 2 Marks

PreparedStatement binds user values separately from SQL syntax. It reduces SQL-injection risk and satisfies the specific JDBC requirement in the rubric. In the finished project, this example should run against a test product or a dedicated demo DAO and must respect the same validation and authorization rules as the main application.

Illustrative JDBC CRUD using PreparedStatement. Place each block inside a method that receives a valid Connection and validated inputs.

```
// CREATE
String insertSql = "INSERT INTO PRODUCT " +
    "(name_en, name_ar, price_sar, stock_quantity, category_id) VALUES (?, ?, ?, ?, ?)";
try (PreparedStatement ps = con.prepareStatement(insertSql)) {
    ps.setString(1, nameEn);
    ps.setString(2, nameAr);
    ps.setBigDecimal(3, priceSar);
    ps.setInt(4, stockQuantity);
    ps.setLong(5, categoryId);
    ps.executeUpdate();
}

// READ
String readSql = "SELECT product_id, name_en, name_ar, price_sar FROM PRODUCT WHERE product_id = ?";
try (PreparedStatement ps = con.prepareStatement(readSql)) {
    ps.setLong(1, productId);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) {
            String englishName = rs.getString("name_en");
            String arabicName = rs.getString("name_ar");
        }
    }
}

// UPDATE
String updateSql = "UPDATE PRODUCT SET name_en = ?, name_ar = ?, price_sar = ? WHERE product_id = ?";
try (PreparedStatement ps = con.prepareStatement(updateSql)) {
    ps.setString(1, nameEn);
    ps.setString(2, nameAr);
    ps.setBigDecimal(3, priceSar);
    ps.setLong(4, productId);
    ps.executeUpdate();
}

// DELETE (use only for an unused test product; production products with history should be deactivated)
String deleteSql = "DELETE FROM PRODUCT WHERE product_id = ?";
try (PreparedStatement ps = con.prepareStatement(deleteSql)) {
    ps.setLong(1, productId);
    ps.executeUpdate();
}
```

```
private static long findCategoryId(Connection connection) throws SQLException {
    String sql = "SELECT id FROM categories ORDER BY id LIMIT 1";
    try (PreparedStatement ps = connection.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        if (!rs.next()) {
            throw new SQLException(reason: "No category exists in nabta_db; cannot create test proc
        }
        return rs.getLong(columnLabel: "id");
    }
}

private static long insertTestProduct(Connection connection, long categoryId, String testName)
        throws SQLException {
    String sql = "INSERT INTO products "
            + "(is_active, created_at, description, image_url, name, price, stock_quantity, "
            + "updated_at, category_id, description_ar, name_ar) "
            + "VALUES (?, CURRENT_TIMESTAMP(6), ?, ?, ?, ?, ?, CURRENT_TIMESTAMP(6), ?, ?, ?)";
    try (PreparedStatement ps = connection.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS))
        ps.setBoolean(parameterIndex: 1, x: true);
        ps.setString(parameterIndex: 2, x: "Temporary JDBC PreparedStatement demonstration product"
        ps.setNull(parameterIndex: 3, Types.VARCHAR);
        ps.setString(parameterIndex: 4, testName);
        ps.setBigDecimal(parameterIndex: 5, new BigDecimal(val: "1.00"));
        ps.setInt(parameterIndex: 6, x: 1);
        ps.setLong(parameterIndex: 7, categoryId);
```

```
private static void updateTestProduct(Connection connection, long productId, String updatedNa
        throws SQLException {
    String sql = "UPDATE products SET name = ?, name_ar = ?, price = ?, "
            + "updated_at = CURRENT_TIMESTAMP(6) WHERE id = ?";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setString(parameterIndex: 1, updatedName);
        ps.setString(parameterIndex: 2, x: "تم تحديث اختبار JDBC");
        ps.setBigDecimal(parameterIndex: 3, new BigDecimal(val: "2.00"));
        ps.setLong(parameterIndex: 4, productId);
        if (ps.executeUpdate() != 1) {
            throw new SQLException(reason: "Expected one test product to be updated.");
        }
    }
}

private static void deleteTestProduct(Connection connection, long productId) throws SQLExcept
    String sql = "DELETE FROM products WHERE id = ?";
    try (PreparedStatement ps = connection.prepareStatement(sql)) {
        ps.setLong(parameterIndex: 1, productId);
        if (ps.executeUpdate() != 1) {
            throw new SQLException(reason: "Expected one test product to be deleted.");
        }
    }
```

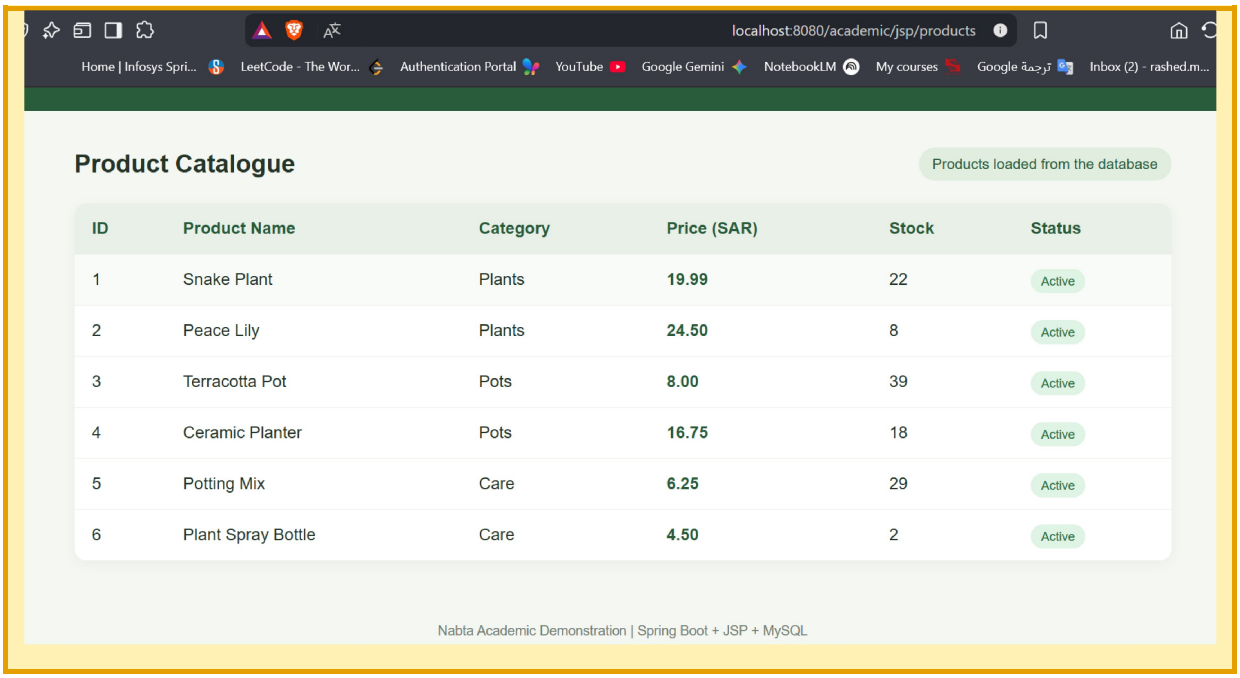
```
>> java -cp $runtimeClasspath com.nabta.backend.academic.JdbcPreparedStatementDemo
INSERT: created temporary test product id=7
SELECT after INSERT: id=7, name=JDBC-DEMO-056f1d15, name_ar=ل‌à ╪ز╪س╪ت╪▓ è╪د è, price=1.
UPDATE: changed temporary product name and price.
SELECT after UPDATE: id=7, name=JDBC-DEMO-056f1d15-UPDATED, name_ar=╪زل‌à ╪ز╪س╪»ل‌è╪س ╪د╪«╪ز
stock=1
DELETE: temporary product deleted.
All four PreparedStatement CRUD steps completed successfully.
ROLLBACK: transaction reverted; no demo product remains.
PS C:\Users\univercity\Desktop\nabta project\nabta_phase1_backend>
```

14.5 GUI using JSP — 1 Mark

JSP (JavaServer Pages) is a server-side technology for generating HTML from Java web applications. For this academic requirement, the project includes a JSP-based web demonstration interface—such as a product list and product form—served by the Java web application. The Android customer application remains built with Ionic and Capacitor; the JSP interface is a separate academic GUI component unless the instructor specifically requires the entire GUI to be JSP.

Illustrative JSP view. Final project must include a working controller/model and compatible JSP/JSTL configuration.

```
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ taglib prefix="c" uri="jakarta.tags.core" %>
<html>
<head><title>Nabta Product Management</title></head>
<body>
  <h2>Nabta - Product Management</h2>
  <table>
    <tr><th>English</th><th>Arabic</th><th>Price (SAR)</th></tr>
    <c:forEach var="product" items="${products}">
      <tr>
        <td><c:out value="${product.nameEn}" /></td>
        <td dir="rtl"><c:out value="${product.nameAr}" /></td>
        <td><c:out value="${product.priceSar}" /></td>
      </tr>
    </c:forEach>
  </table>
</body>
</html>
```

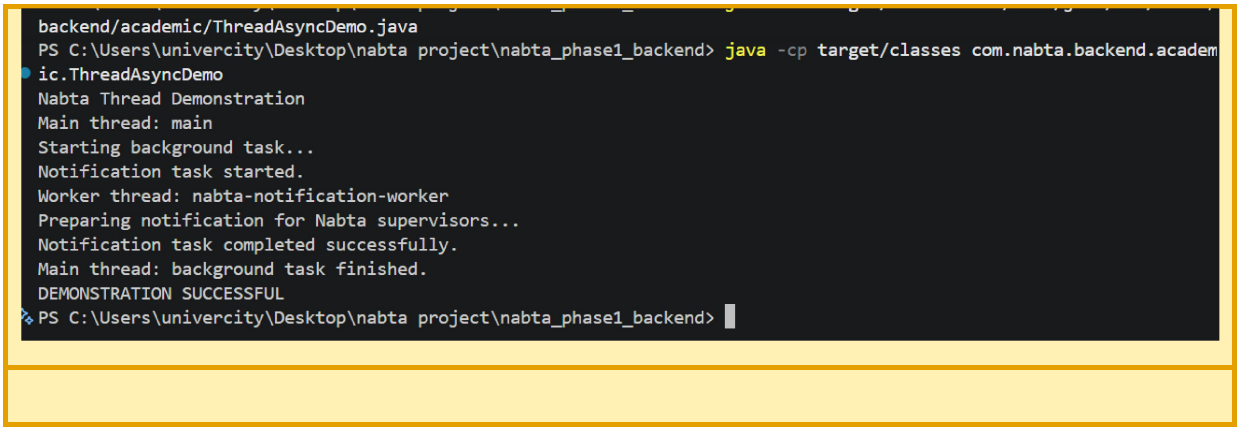


14.6 Thread — 1 Mark

A thread is a separate path of execution. A controlled background task can be used to demonstrate non-blocking processing, such as preparing a notification after an order has been persisted. The production code should use managed asynchronous execution or another safe task mechanism rather than creating unbounded threads for each request.

Illustrative thread demonstration. In a Spring application, prefer a configured executor or @Async for production workloads.

```
Runnable notifySupervisors = () -> {
    notificationService.notifyRegisteredSupervisors(orderId);
};
Thread notificationThread = new Thread(notifySupervisors, "nabta-order-notification");
notificationThread.start();
```



14.7 Viva Voce — 1 Mark

The student should be able to explain the purpose of each layer, describe how PreparedStatement works, distinguish JSP from the Ionic frontend, explain the inheritance example, discuss collection choices, identify exception-handling paths, and describe how the database relationships enforce consistency.

15. Testing Plan and Demonstration

The tests below define acceptance checks for Nabta. The actual result column must be filled after running each test. The report must not claim an item passed until a real test has been observed and its evidence captured.

ID	Test	Expected result	Actual result
T-01	Customer registration/login	Valid login succeeds; invalid credentials are rejected.	[FILL: PASS/FAIL + evidence]
T-02	Phone validation	Seven or more digits accepted; letters, spaces, and symbols rejected.	[FILL: PASS/FAIL + evidence]
T-03	Bilingual catalogue	Both English and Arabic names appear; order adapts to selected language.	[FILL: PASS/FAIL + evidence]
T-04	Product CRUD/image	Owner creates/edits/deactivates a test product and image remains available.	[FILL: PASS/FAIL + evidence]
T-05	Cart quantity	Add confirmation appears and +/- changes the quantity and totals correctly.	[FILL: PASS/FAIL + evidence]
T-06	Preferences/address	Add/remove preference; save/select address; persistence checked after refresh.	[FILL: PASS/FAIL + evidence]
T-07	COD checkout	Order saved with COD and correct SAR total; no online option presented as active.	[FILL: PASS/FAIL + evidence]
T-08	Role access	Customer cannot access Owner/Supervisor/Delivery-only endpoints.	[FILL: PASS/FAIL + evidence]
T-09	New-order notification	Owner and registered Supervisors receive notification for new order.	[FILL: PASS/FAIL + evidence]

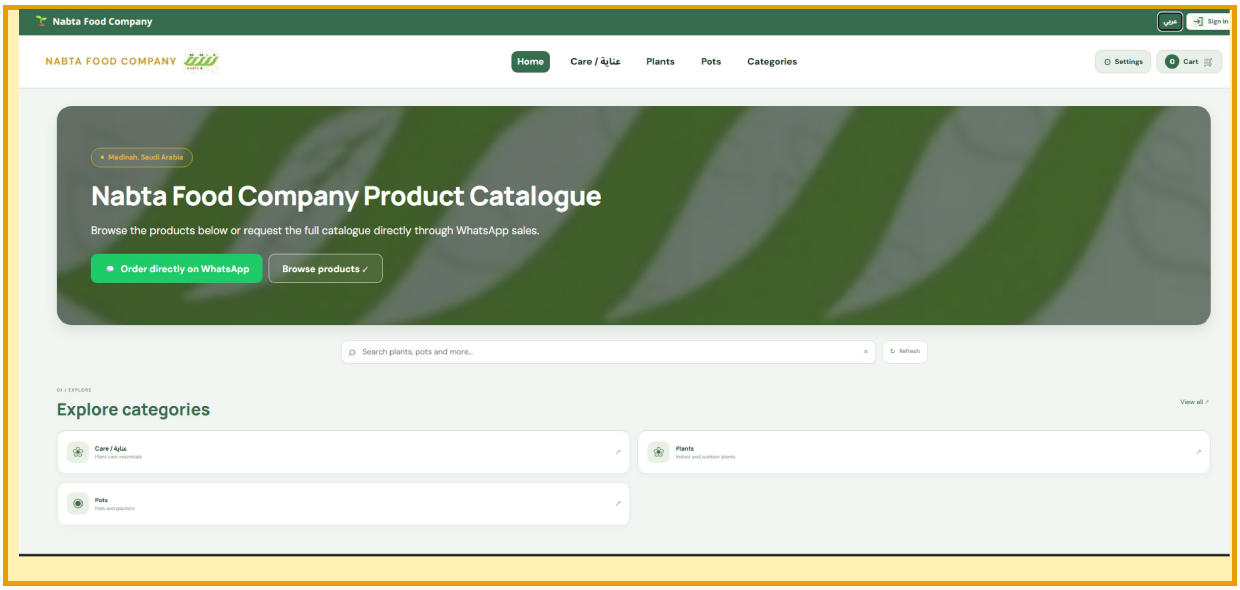
ID	Test	Expected result	Actual result
T-10	Delivery assignment	Assigned Delivery user receives notification and sees only authorized assignments.	[FILL: PASS/FAIL + evidence]
T-11	Partial dispatch	Selected quantities are dispatched; undelivered remainder is cancelled per policy.	[FILL: PASS/FAIL + evidence]
T-12	Order audit	Status changes show responsible user, old/new status, and timestamp.	[FILL: PASS/FAIL + evidence]
T-13	Owner statistics	Cancelled, delivered, partially delivered, and product counts match test data.	[FILL: PASS/FAIL + evidence]
T-14	MySQL SQL objects	Queries, function, procedure, and trigger execute in the test database.	[FILL: PASS/FAIL + evidence]
T-15	JSP GUI/JDBC CRUD	JSP page loads and PreparedStatement CRUD works on test records.	[FILL: PASS/FAIL + evidence]
T-16	Android network	Installed app loads the remote deployed API without depending on the developer PC IP.	[FILL: PASS/FAIL + evidence]

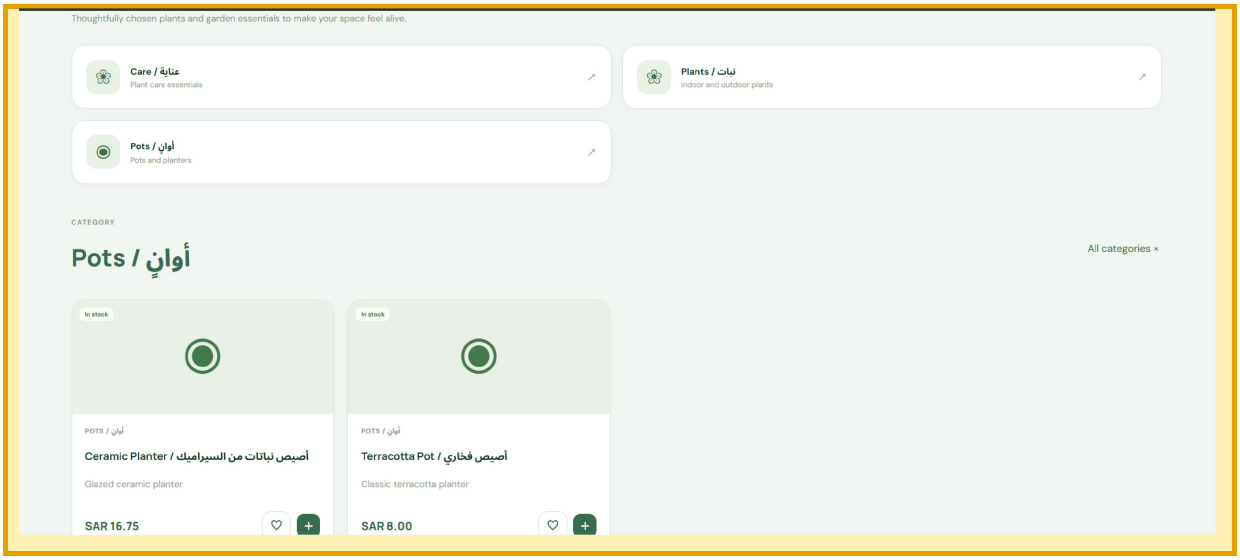
15.1 Database Screenshots

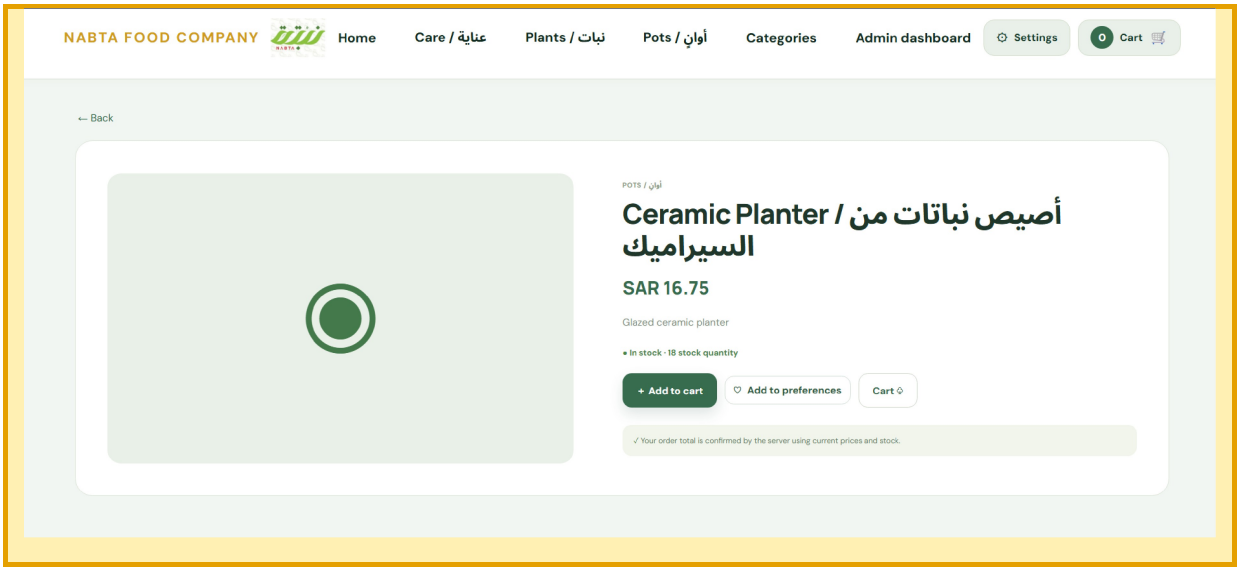
Insert the real screenshots after the relevant SQL commands have been executed successfully. Use a test database or test records for operations that modify data.

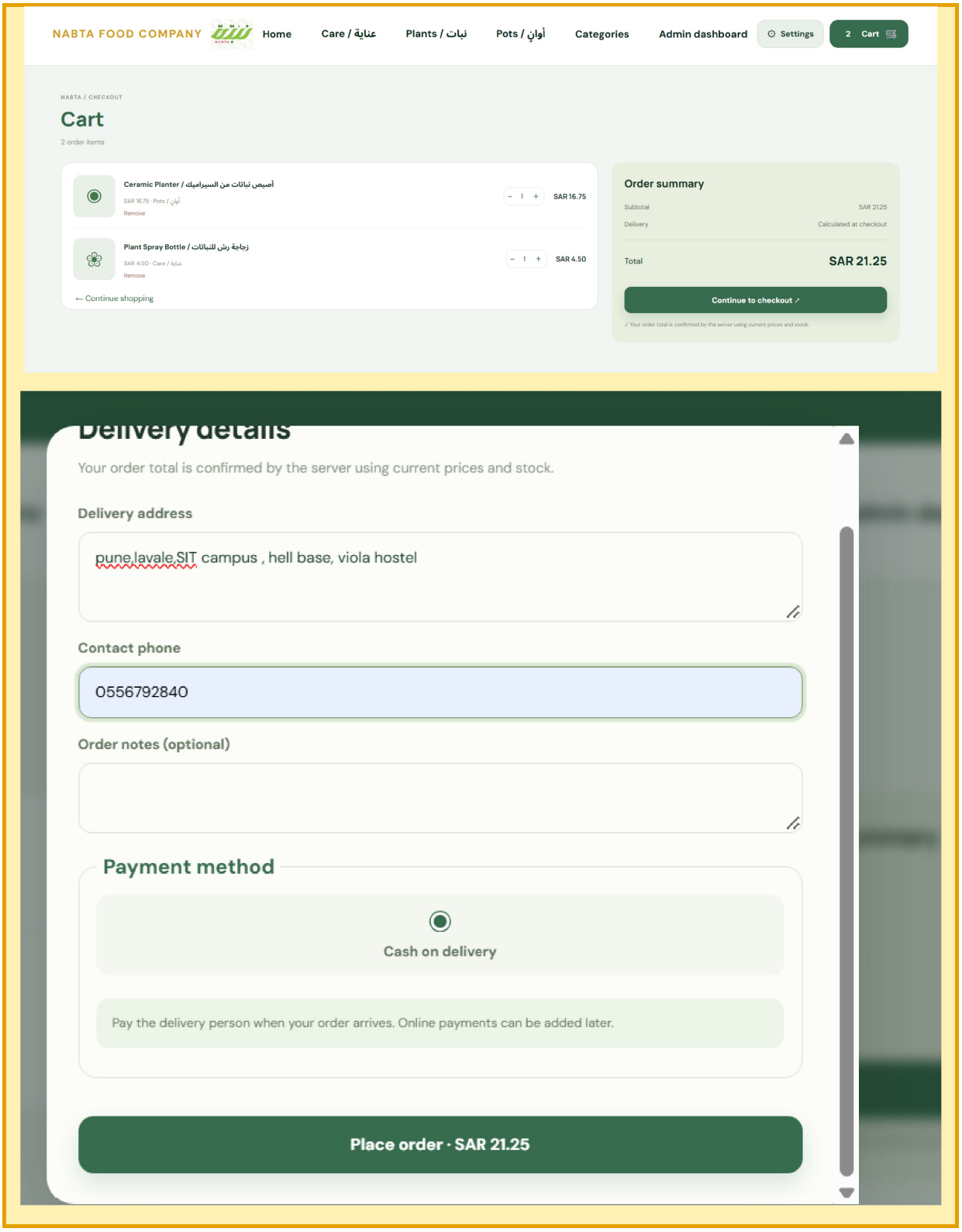
Evidence reference: Successful execution results for the SQL function, stored procedure, and trigger are provided in Section 13 and the corresponding DB-04, DB-05, and DB-06 screenshots. Additional screenshots are omitted to avoid duplication.

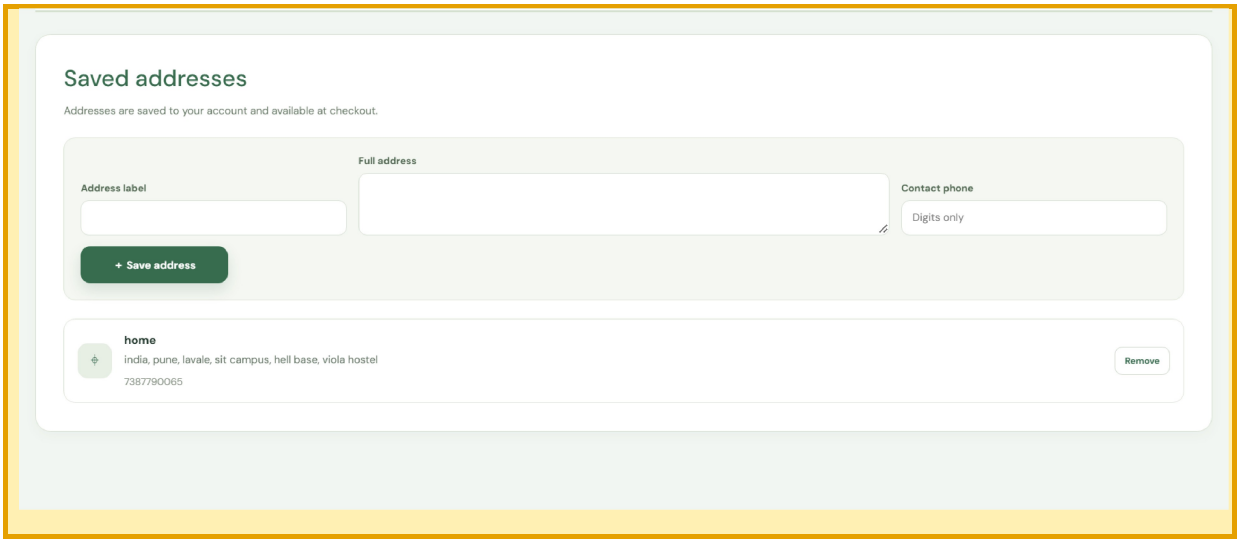
15.2 Application Demonstration Screenshots

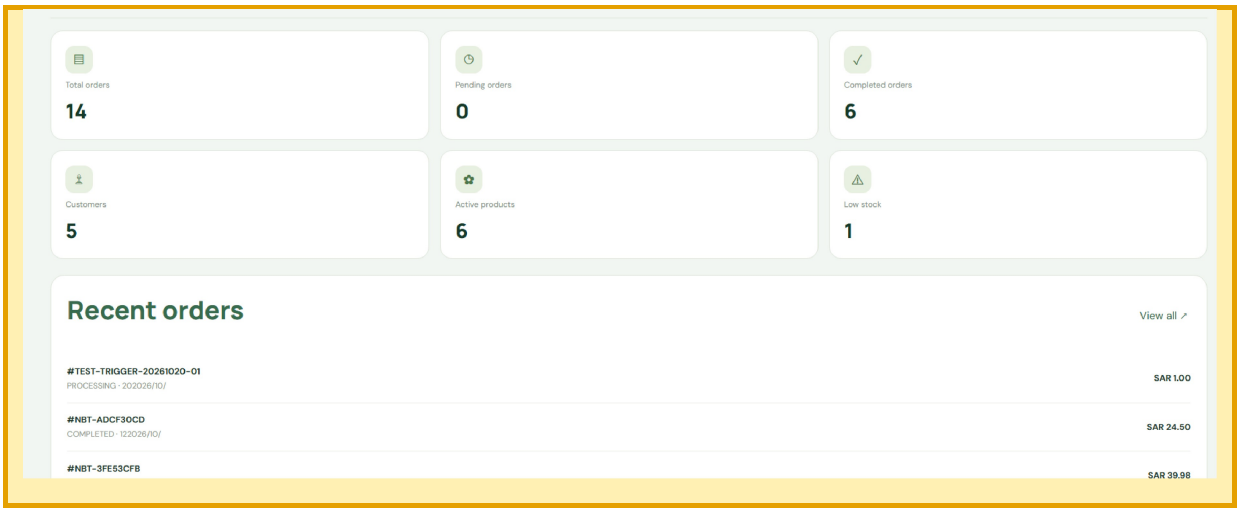


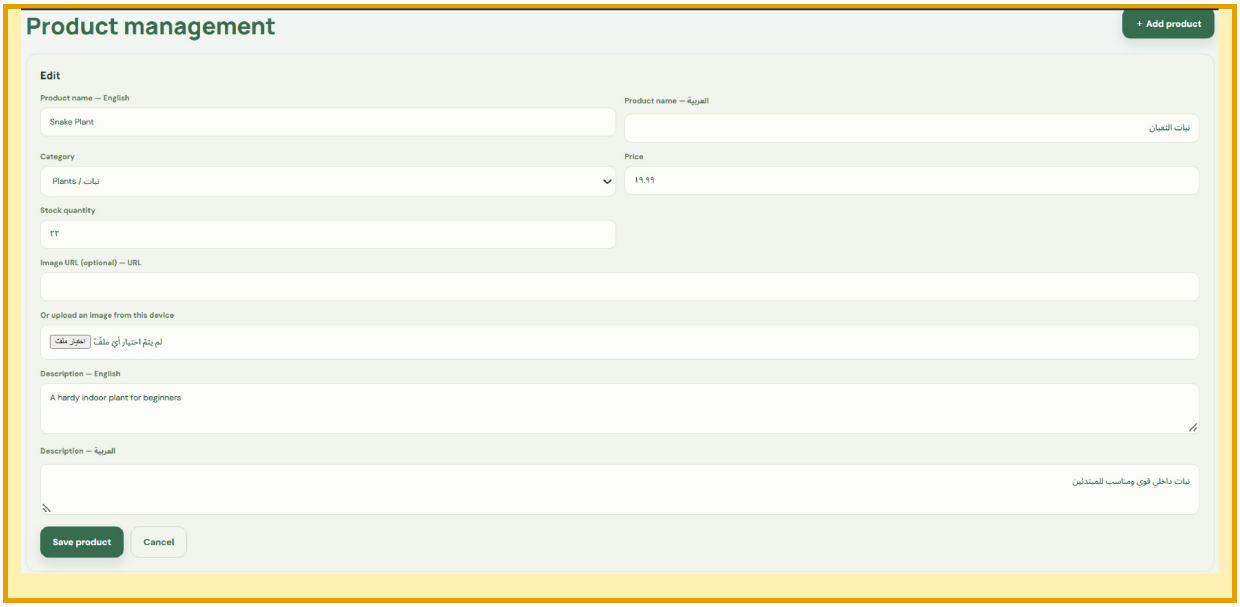

Thoughtfully chosen plants and garden essentials to make your space feel alive.
Care / عناية
Plant care essentials
Plants / نبات
Indoor and outdoor plants
Pots / أوانٍ
Pots and planters
CATEGORY
Pots / أوانٍ
All categories ×
In stock
POTS / أوانٍ
Ceramic Planter / أصيص نباتات من السيراميك
Glazed ceramic planter
SAR 16.75
In stock
POTS / أوانٍ
Terracotta Pot / أصيص فخاري
Classic terracotta planter
SAR 8.00

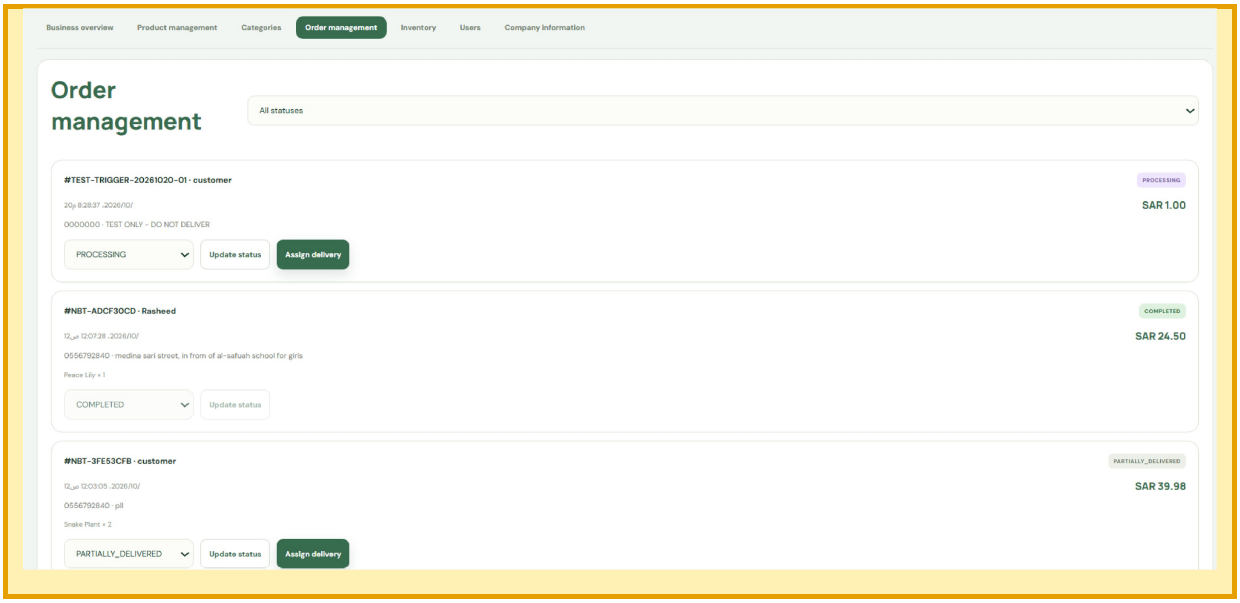

NABTA FOOD COMPANY
Home
Care / عناية
Plants / نبات
Pots / أوانٍ
Categories
Admin dashboard
Settings
0
Cart
← Back
POTS / أوانٍ
Ceramic Planter / أصيص نباتات من السيراميك
SAR 16.75
Glazed ceramic planter
• In stock · 18 stock quantity
+ Add to cart
Add to preferences
Cart
✓ Your order total is confirmed by the server using current prices and stock.

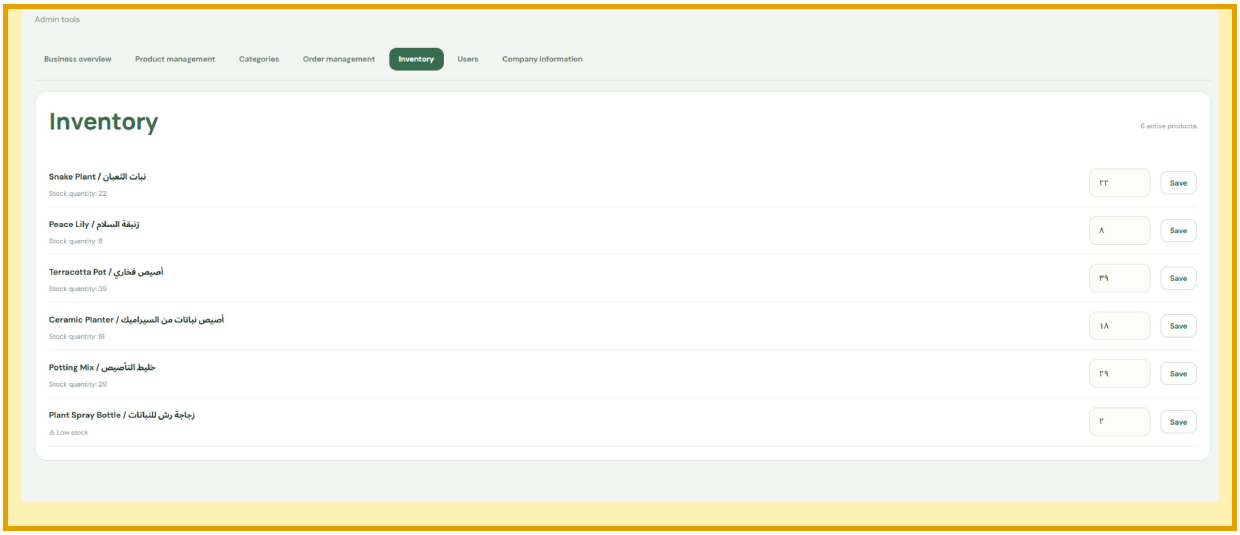

NABTA FOOD COMPANY
Home
Care / عناية
Plants / نبات
Pots / أوانٍ
Categories
Admin dashboard
Settings
2 Cart
NABTA / CHECKOUT
Cart
2 order items
Ceramic Planter / أصيص نباتات من السيراميك
SAR 16.75 · Pots / أوانٍ
Remove
SAR 16.75
Plant Spray Bottle / زجاجة رش للنباتات
SAR 4.50 · Care / عناية
Remove
SAR 4.50
← Continue shopping
Order summary
Subtotal
SAR 2125
Delivery
Calculated at checkout
Total
SAR 21.25
Continue to checkout ↗
✓ Your order total is confirmed by the server using current prices and stock.
Delivery details
Your order total is confirmed by the server using current prices and stock.
Delivery address
pune,lavale,SIT campus , hell base, viola hostel
Contact phone
0556792840
Order notes (optional)
Payment method
Cash on delivery
Pay the delivery person when your order arrives. Online payments can be added later.
Place order · SAR 21.25

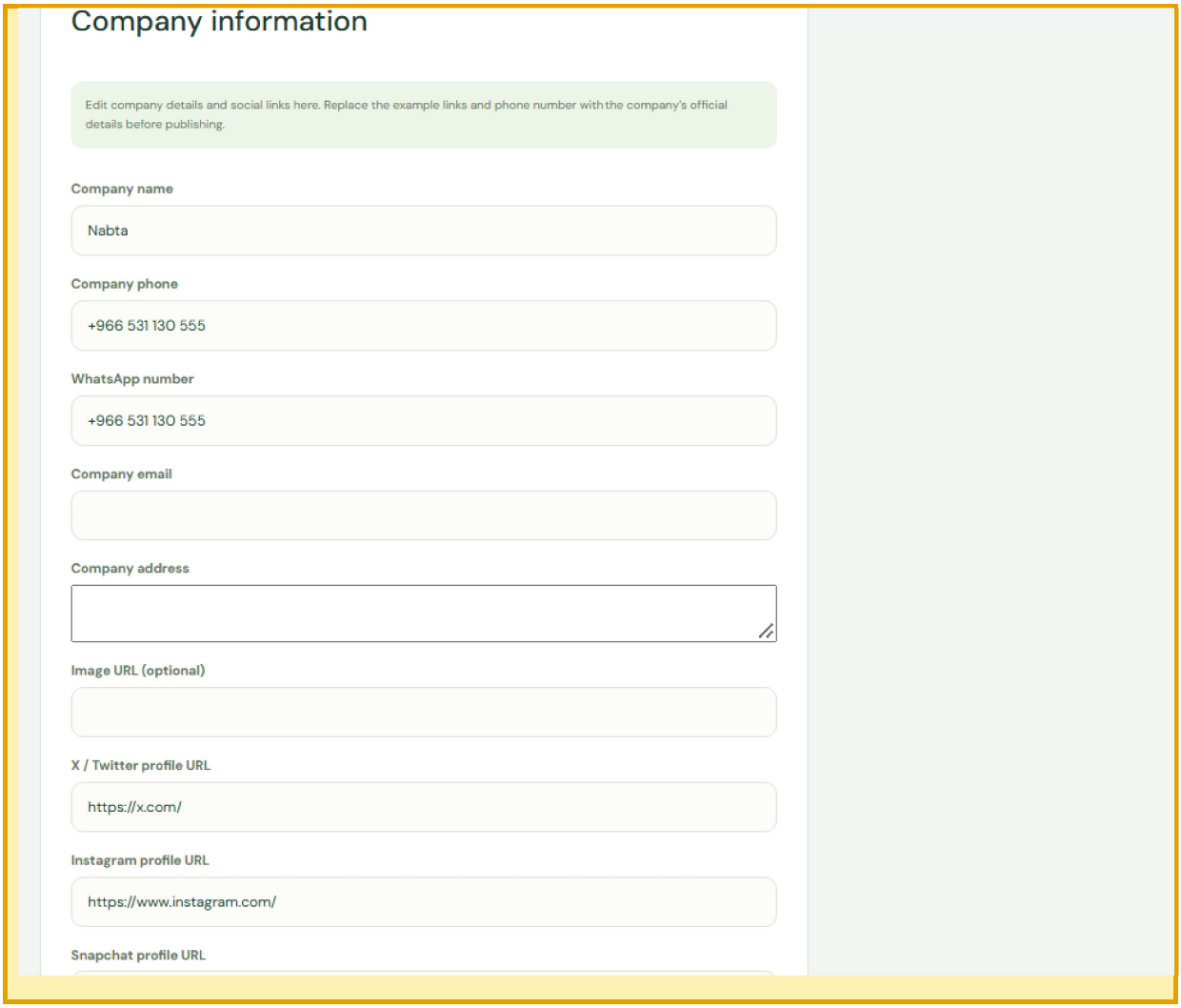

Saved addresses
Addresses are saved to your account and available at checkout.
Full address
Address label
Contact phone
Digits only
+ Save address
home
india, pune, lavale, sit campus, hell base, viola hostel
7387790065
Remove

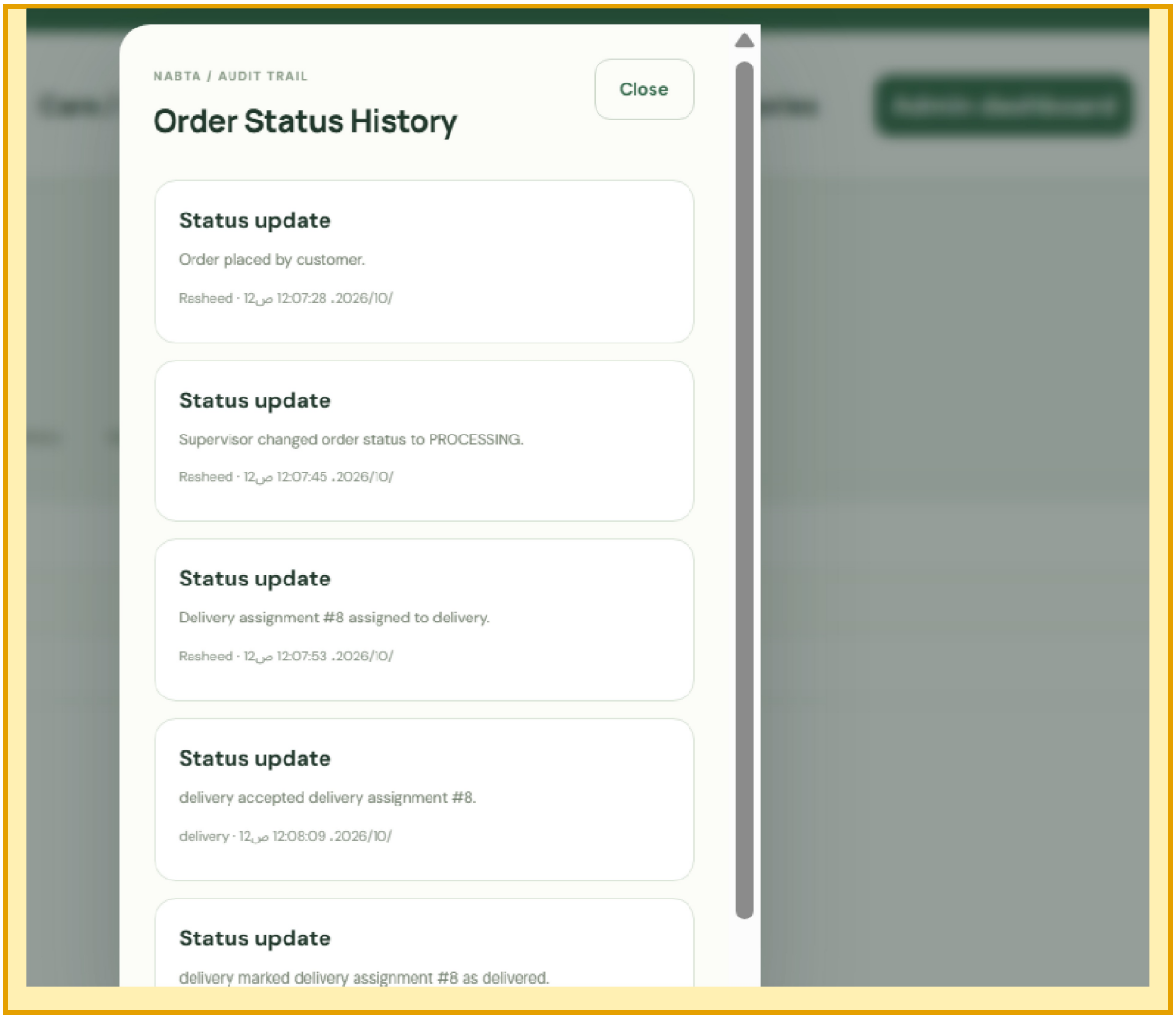

Total orders
14
Pending orders
0
Completed orders
6
Customers
5
Active products
6
Low stock
1
Recent orders
View all ↗
#TEST-TRIGGER-20261020-01
PROCESSING · 202026/10/
SAR 1.00
#NBT-ADCF30CD
COMPLETED · 122026/10/
SAR 24.50
#NBT-3FE53CFB
SAR 39.98


Product management
+ Add product
Edit
Product name — English
Snake Plant
Product name — العربية
نبات الثعبان
Category
Plants / نبات
Price
١٩.٩٩
Stock quantity
٢٢
Image URL (optional) — URL
Or upload an image from this device
اختيار ملف
لم يتم اختيار أي ملف
Description — English
A hardy indoor plant for beginners
Description — العربية
نبات داخلي قوي ومناسب للمبتدئين
Save product
Cancel


Business overview
Product management
Categories
Order management
Inventory
Users
Company information
Order management
All statuses
#TEST-TRIGGER-20261020-01 · customer
PROCESSING
SAR 1.00
0000000 · TEST ONLY – DO NOT DELIVER
PROCESSING
Update status
Assign delivery
#NBT-ADCF30CD · Rasheed
COMPLETED
SAR 24.50
0556792840 · medina sari street, in from of al-safuah school for girls
Peace Lily × 1
COMPLETED
Update status
#NBT-3FE53CFB · customer
PARTIALLY_DELIVERED
SAR 39.98
0556792840 · pll
Snake Plant × 2
PARTIALLY_DELIVERED
Update status
Assign delivery


Admin tools
Business overview
Product management
Categories
Order management
Inventory
Users
Company information
Inventory
6 active products
Snake Plant / نبات الثعبان
Stock quantity: 22
Save
Peace Lily / زنبقة السلام
Stock quantity: 8
Save
Terracotta Pot / أصيص فخاري
Stock quantity: 39
Save
Ceramic Planter / أصيص نباتات من السيراميك
Stock quantity: 18
Save
Potting Mix / خليط التأصيص
Stock quantity: 29
Save
Plant Spray Bottle / زجاجة رش للنباتات
⚠ Low stock
Save


Company information
Edit company details and social links here. Replace the example links and phone number with the company's official details before publishing.
Company name
Nabta
Company phone
+966 531 130 555
WhatsApp number
+966 531 130 555
Company email
Company address
Image URL (optional)
X / Twitter profile URL
https://x.com/
Instagram profile URL
https://www.instagram.com/
Snapchat profile URL



16. Limitations and Future Enhancements

- Online payment provider integration is not active. The current available checkout method is Cash on Delivery until a provider is approved, configured, and payment verification is tested.
- Public deployment requires production hosting, HTTPS, secure secrets, persistent image storage, database backups, and production security configuration.
- The documentation must be reconciled with the final MySQL schema and the actual Java classes before submission.
- JSP is an academic web-GUI component distinct from the existing Ionic/Capacitor Android client; the instructor's scope requirement should be confirmed.
- Real SMS verification, if required for mobile numbers, needs an external verification provider; the current requirement described here is format validation only.
- Future development may include a provider-backed online payment gateway, refund workflow, stronger delivery tracking, and more advanced reporting.

17. Conclusion

Nabta combines a bilingual commerce and order-management system that combines a mobile application, Java backend, and MySQL database. Its proposed data model separates customer accounts, catalogue information, carts, orders, delivery assignments, and audit history to improve integrity and traceability.

English and Arabic product/category fields support bilingual presentation, while order-item snapshots preserve historical purchase information.

The project covers the database topics required by the DBMS rubric, including ER/EER modelling, relational mapping, functional dependencies, normalization, SQL queries, functions, procedures, triggers, and Codd's rules. It also includes a plan to demonstrate inheritance, collections, exception handling, JDBC PreparedStatement CRUD, JSP GUI, and threading for the Java rubric. Implementation evidence and actual test outcomes should be completed in Section 15 before the report is submitted.

Appendix A. Screenshot Capture Guide

This appendix tells the student what to capture later. Every orange screenshot marker in the report uses the corresponding identifier. Capture only the requested region, keep text readable, and do not include passwords, API keys, Firebase service-account files, or unrelated personal information.

ID	Where	How to capture
DB-01	MySQL Workbench	Open MySQL Workbench, connect to the local/test Nabta database, expand SCHEMAS → nabta_db → Tables, and use Win+Shift+S to capture the table list.
DB-02	MySQL Workbench	Open a query tab and run DESCRIBE PRODUCT; and DESCRIBE CUSTOMER_ORDER; or open the table structure tab. Capture columns and key information.
DB-03	MySQL Workbench	Run Query 1 or Query 4 from Section 13. Show the SQL and result grid in the same capture; use test/sample data only.
DB-04	MySQL Workbench	Run SELECT fn_order_subtotal(<valid_test_order_id>); and show the function result.
DB-05	MySQL Workbench	Run CALL sp_order_status_summary(); and show the returned grouped counts.
DB-06	MySQL Workbench	Create/use a test order, change its status using the test workflow, then query ORDER_STATUS_HISTORY for that order. Never alter a real order solely for a screenshot.
JAVA-01	VS Code / IDE	Open the Java DAO file that uses PreparedStatement. Capture the CRUD methods and the relevant test output without showing secrets.
JAVA-02	Browser	Run the JSP demonstration application, open its product list or form, and capture the browser page. Include a local URL only if it contains no sensitive token.
JAVA-03	Android / IDE	Trigger a test notification or run the thread demonstration. Capture the Android notification or successful output, hiding device identifiers if needed.
APP-01	Nabta application	Open the home page at the top and use Win+Shift+S on Windows or the phone screenshot buttons on Android.
APP-02	Nabta application	Choose a product card where both English and Arabic names and the SAR price are readable.
APP-03	Nabta application	Open a product's detail screen and show the preference add/remove control.
APP-04	Nabta application	Add a test item to the cart and open checkout. Show quantity, SAR total, and COD choice; do not submit a real customer order.
APP-05	Nabta application	Open the profile/address area using a test account. Mask or crop email, phone, and street details.
APP-06	Nabta application	Log in as Owner and open the dashboard. Use clearly labelled test data if possible.
APP-07	Nabta application	Log in as Owner and open product edit/add screen. Show both name fields and image upload controls, without needing to show private file paths.
APP-08	Nabta application	Open account management as Owner and show the role choices; crop unrelated customer personal data.
APP-09	Nabta application	Log in as Supervisor and open a test order. Show status and dispatch options.
APP-10	Nabta application	Log in as Delivery and show an assigned test delivery and its accept/complete controls.
APP-11	Nabta application	As Owner, open the order audit/history view for a test order and show the actor, old/new state, and timestamp.
APP-12	Nabta application	Open the employee activity view as Owner and capture relevant actions with timestamps.
APP-13	Android notification shade	Put Nabta in the background, trigger a test new-order or assignment event, pull down the notification shade, and screenshot the received alert.

Appendix B. Viva Preparation Questions

Question	Short answer
1. What problem does Nabta solve?	It centralizes product catalogue, customer ordering, employee order processing, and delivery tracking in a shared database-backed application.
2. What is the role of the primary key?	It uniquely identifies each row and prevents ambiguous references.
3. Why are foreign keys used?	They preserve valid relationships between parent and child records.
4. What is the difference between ER and EER?	ER shows entities, attributes, and relationships; EER adds enhanced concepts such as specialization/generalization.

Question	Short answer
5. What is a functional dependency?	X → Y means that a value of X uniquely determines the value of Y.
6. Why normalize a database?	To reduce unnecessary redundancy and insertion, update, and deletion anomalies.
7. Why is DECIMAL used for money?	It represents fixed decimal precision more safely than binary floating-point types.
8. What is PreparedStatement?	A JDBC object that binds parameter values separately from SQL syntax, helping prevent SQL injection.
9. What is inheritance?	A class can extend another class to reuse or specialize its behavior.
10. What are Java collections?	Data structures such as List, Set, and Map used to manage groups of objects.
11. Why is exception handling needed?	It handles errors predictably and prevents internal details from leaking to the user.
12. What is JSP?	A server-side Java technology for generating dynamic web pages.
13. How is JSP different from the Ionic frontend?	JSP renders web pages on the Java server; Ionic/Capacitor builds the customer-facing hybrid mobile client.
14. What is a thread?	An independent path of execution within a process.
15. Why track order-status history?	It records who changed an order, the previous and new status, and when the change occurred.
16. How does partial delivery work?	Each assignment records selected order-item quantities; delivered and cancelled quantities are tracked without overwriting the original order quantity.
17. Why is payment-provider integration marked future work?	No provider is connected, so COD is the active checkout method and online payment success must not be simulated.
18. What does Codd's Information Rule say?	All information in a relational database is represented explicitly as values in tables.